

NAGARJUNA COLLEGE
of Information Technology

Lab Report
Of
Compiler Design and Construction
CSC 365

Submitted To
Nagarjuna College of IT
(Affiliated to Tribhuwan University)

Submitted by	:	DIPESH SHRESTHA
College Roll No	:	08
Program	:	BSc.CSIT
Semester	:	Third (3rd)
Registration Number	:	5-2-429-21-2021
Symbol No	:	28985 | 078

List of Lab Works

LAB. No	TITLE/QUESTION	Date of Submission	Signature
1	PREFIXES, SUFFIXES AND SUBSTRING		
2	IDENTIFIER & KEYWORD		
3	IMPLEMENTATION OF DFA		
4	IMPLEMENTATION OF REGULAR EXPRESSION		
5	COMMENT IN C		
6	COMPUTATION OF FIRST		
7	COMPUTATION OF FOLLOW		
8	IDENTIFIER VALIDATION		
9	LEXICAL ANALYSIS		
10	TYPE CHECKING		
11	SHIFT REDUCE PARSER		

LAB. No	TITLE/QUESTION	Date of Submission	Signature
12	IMPLEMENTATION OF SYMBOL TABLE		
13	IMPLEMENTATION OF SLR(1) GRAMMER		
14	IMPLEMENTATION OF LL(1) GRAMMER		
15	IMPLEMENTATION OF BRUTE FORCE GRAMMER		

LAB: 1

TITLE: PREFIXES, SUFFIXES AND SUBSTRING

OBJECTIVE: Write a program to find prefixes, suffixes and substring from given string.

Prefix: The string which is obtained after removing 0 or more trailing symbols.
Suffix: The string which is obtained after removing 0 or more leading symbols.
Substring: The string which is obtained after removing 0 or more trailing or leading symbols.

SOURCE CODE:

```
#include<stdio.h>
#include<string.h>
void find_prefix(char string[]);
void find_suffix(char string[]);
void find_substring(char string[],int,int);
int main()
{
char string[20];
int i,j;
printf("\n Enter a string\t");
gets(string);
printf("\n Prefixes:");
find_prefix(string);
printf("\n Suffixes");
find_suffix(string);
printf("\n Enter i and j for substring: ");
scanf("%d%d",&i,&j);
find_substring(string,i,j);
printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 01");
return 0;
}
void find_prefix(char string[])
{
int i,j;
char prefix[20];
for(i=strlen(string);i>=0;i--)
{
for( j = 0; j<i;j++)
    {
     prefix[j]= string[j];
    }
    prefix[j]='\0';
    printf("\n %s",prefix);
 }
}
```

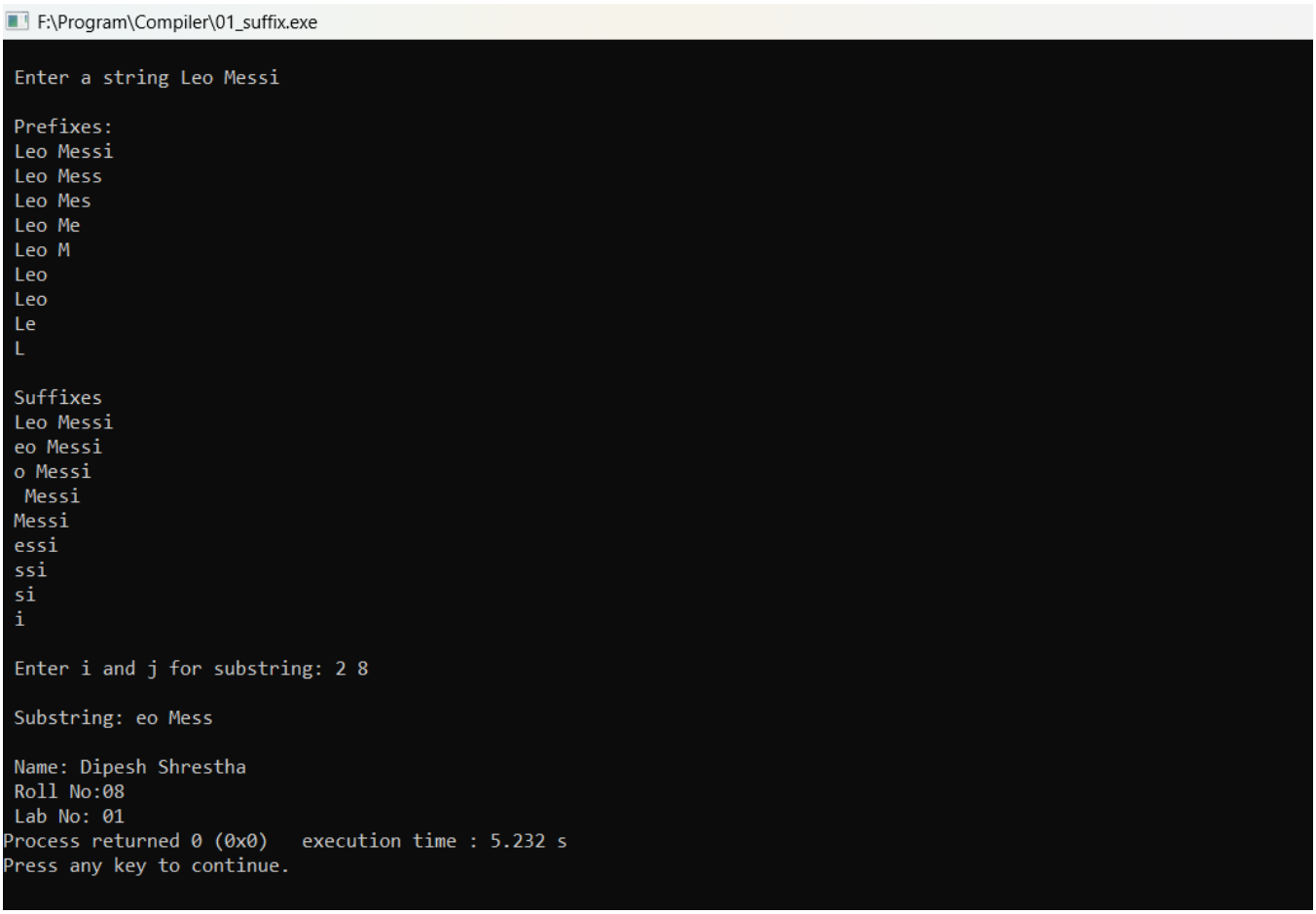
```
void find_suffix(char string[])
{
 int i,j,k;
 char suffix[20];
 for(i=0;i<=strlen(string);i++)
 {
  k = i;
  for( j = 0; j<strlen(string);j++)
    {
     suffix[j]= string[k];
     k++;
    }
    suffix[j]='\0';
    printf("\n %s",suffix);
 }
}

void find_substring(char string[],int x, int y)
{
 char substr[20];

 int k=0;
 for(int i=x-1;i<y;i++)
 {
      substr[k]=string[i];
      k++;
 }
 substr[k]='\0';
 printf("\n Substring: %s",substr);

}
```

OUTPUT


F:\Program\Compiler\01_suffix.exe
Enter a string Leo Messi
Prefixes:
Leo Messi
Leo Mess
Leo Mes
Leo Me
Leo M
Leo
Leo
Le
L
Suffixes
Leo Messi
eo Messi
o Messi
Messi
Messi
essi
ssi
si
i
Enter i and j for substring: 2 8
Substring: eo Mess
Name: Dipesh Shrestha
Roll No:08
Lab No: 01
Process returned 0 (0x0) execution time : 5.232 s
Press any key to continue.

LAB: 2

TITLE: IDENTIFIER & KEYWORD

OBJECTIVE: Write a program to validate C identifiers and keywords.

C identifiers: These are the names of variables, functions, arrays, structures and pointers etc. The first character of C identifiers must be letter or underscore and remaining characters might be letters, digits or underscore.
Keywords: These are the reserved words having predefined meaning in the language. There are 32 keywords in C. They cannot be used as identifiers

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
enum states { q0, q1, qd };
char keyword[32][10] = {
    "auto", "break", "case", "char", "const", "continue", "default", "do",
    "double", "else", "enum", "extern", "float", "for", "goto", "if",
    "int", "long", "register", "return", "short", "signed", "sizeof", "static",
    "struct", "switch", "typedef", "union", "unsigned", "void", "volatile", "while"
};
int isKeyword(char str[]) {
    for (int i = 0; i < 32; i++) {
        if (strcmp(str, keyword[i]) == 0)
            return 1;
    }
    return 0;
}

int isValidIdentifier(char str[]) {
    enum states curr_state = q0;
    int i = 0;
    char ch;

    while ((ch = str[i]) != '\0') {
        switch (curr_state) {
            case q0:
                if (isalpha(ch) || ch == '_')
                    curr_state = q1;
                else
                    curr_state = qd;
                break;
```

```
            case q1:
                if (isalnum(ch) || ch == '_')
                    curr_state = q1;
                else
                    curr_state = qd;
                break;

            case qd:
                return 0;
        }
        i++;
    }

    return (curr_state == q1);
}

int main() {
    char str[50];

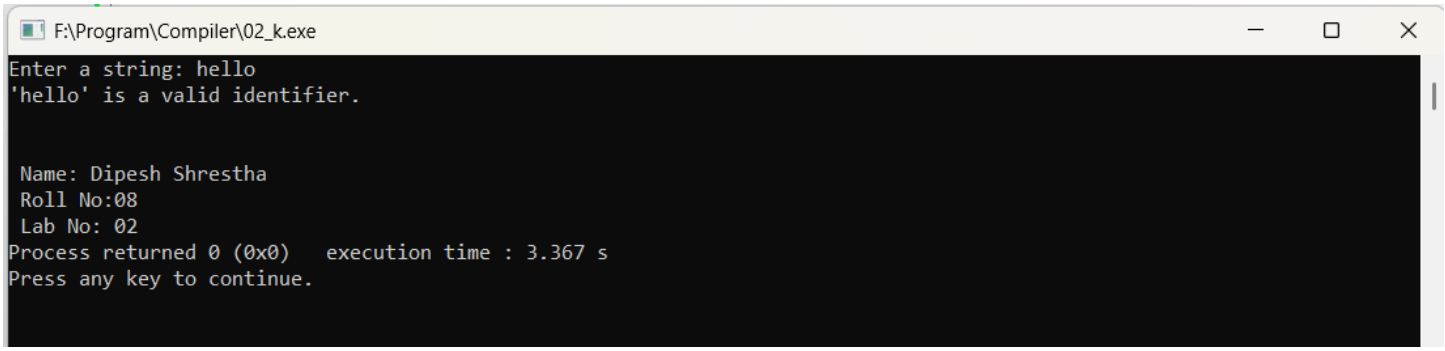
    printf("Enter a string: ");
    scanf("%s", str);

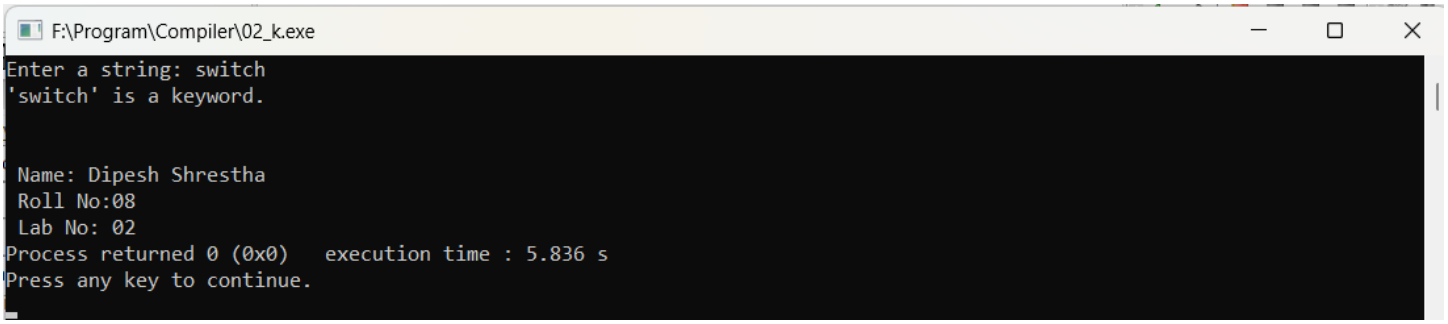
    if (isKeyword(str)) {
        printf("'%s' is a keyword.\n", str);
    } else if (isValidIdentifier(str)) {
        printf("'%s' is a valid identifier.\n", str);
    } else {
        printf("'%s' is not a valid identifier.\n", str);
    }

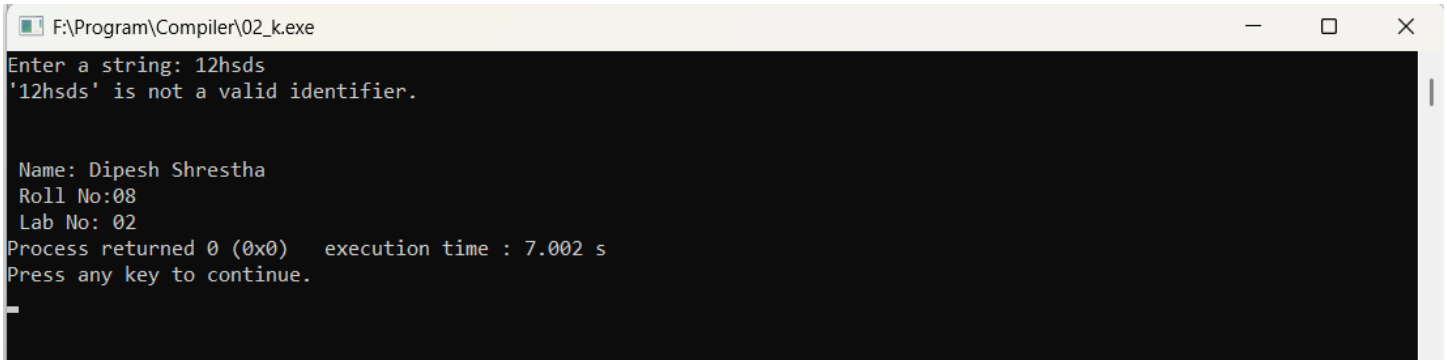
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 02");

    return 0;
}
```

OUTPUT


F:\Program\Compiler\02_k.exe
Enter a string: hello
'hello' is a valid identifier.
Name: Dipesh Shrestha
Roll No:08
Lab No: 02
Process returned 0 (0x0) execution time : 3.367 s
Press any key to continue.


F:\Program\Compiler\02_k.exe
Enter a string: switch
'switch' is a keyword.
Name: Dipesh Shrestha
Roll No:08
Lab No: 02
Process returned 0 (0x0) execution time : 5.836 s
Press any key to continue.


F:\Program\Compiler\02_k.exe
Enter a string: 12hsds
'12hsds' is not a valid identifier.
Name: Dipesh Shrestha
Roll No:08
Lab No: 02
Process returned 0 (0x0) execution time : 7.002 s
Press any key to continue.

LAB: 3

TITLE: IMPLEMENTATION OF DFA

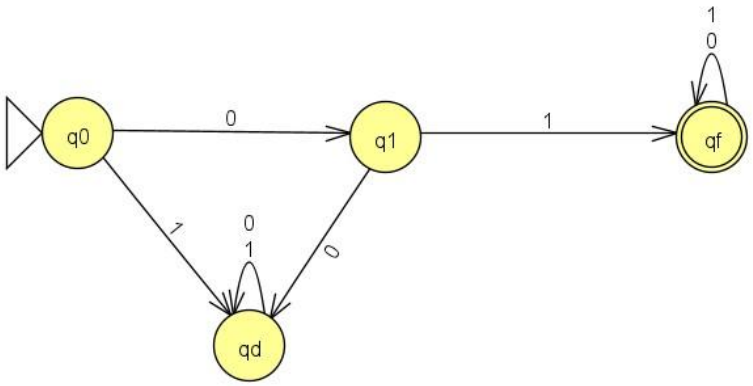
OBJECTIVE: Write a C program to implement following DFA's over alphabet L={ 0,1}

i. The DFA that accepts all strings that start with 01.

- Transition Table

State	Inputs	
	0	1
q0	q1	qd
q1	qd	qf
qd	qd	qd
qf	qf	qf

- Transition diagram



SOURCE CODE:

```
#include <stdio.h>

enum states { q0, q1, qf, qd };

int main() {
    char input[20];
    enum states curr_state = q0;
    int i = 0;

    printf("\nEnter a binary string\t");
    gets(input);
```

```
    char ch = input[i];
    while (ch != '\0') {
        switch (curr_state) {
            case q0:
                if (ch == '0')
                    curr_state = q1;
                else
                    curr_state = qd;
                break;

            case q1:
                if (ch == '1')
                    curr_state = qf;
                else
                    curr_state = qd;
                break;

            case qf:
                if (ch == '0' || ch == '1')
                    curr_state = qf;
                break;

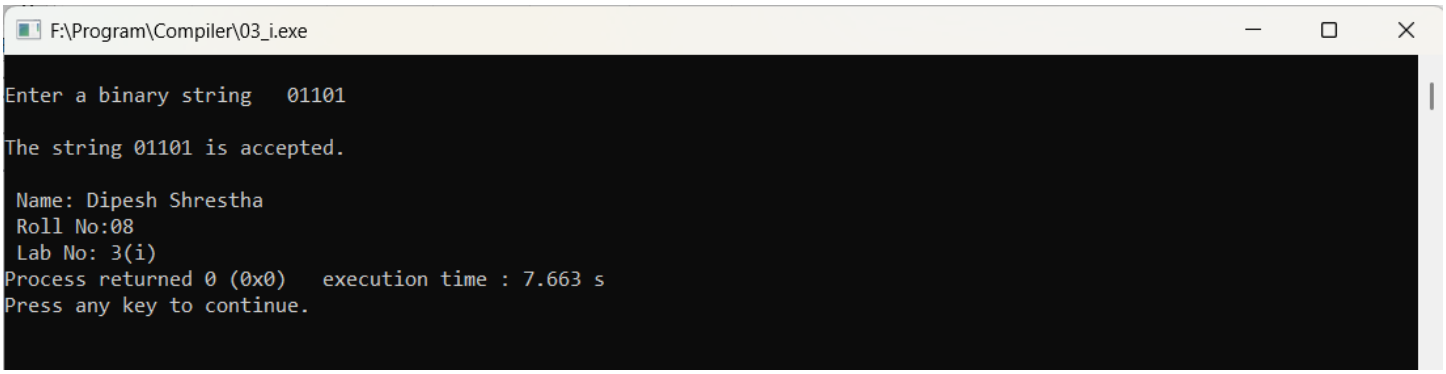
            case qd:
                if (ch == '0' || ch == '1')
                    curr_state = qd;
                break;
        }
        ch = input[++i];
    }

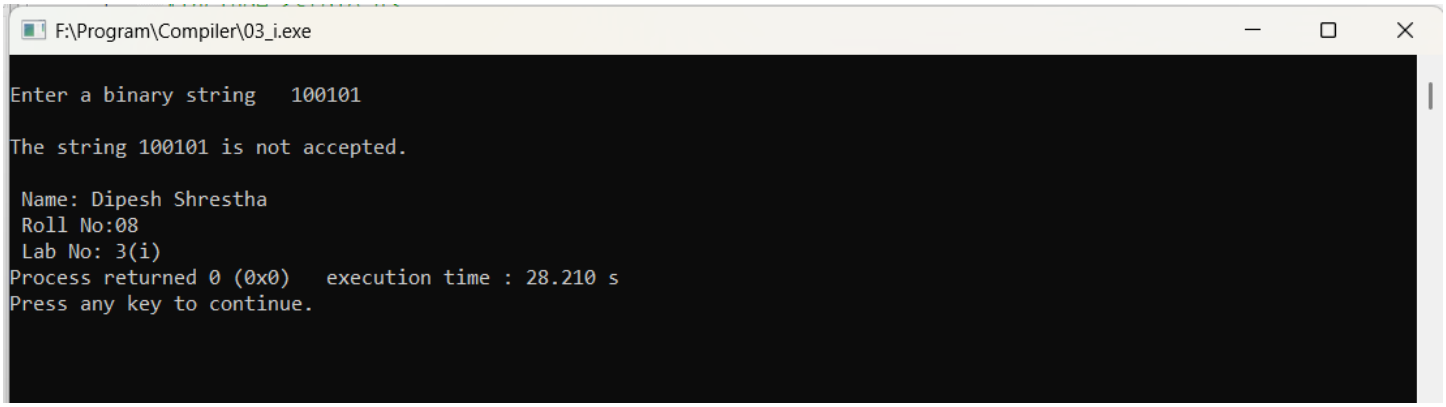
    if (curr_state == qf)
        printf("\nThe string %s is accepted.", input);
    else
        printf("\nThe string %s is not accepted.", input);

    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 3(i)");

    return 0;
}
```

OUTPUT:


F:\Program\Compiler\03_i.exe
Enter a binary string 01101
The string 01101 is accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(i)
Process returned 0 (0x0) execution time : 7.663 s
Press any key to continue.


F:\Program\Compiler\03_i.exe
Enter a binary string 100101
The string 100101 is not accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(i)
Process returned 0 (0x0) execution time : 28.210 s
Press any key to continue.

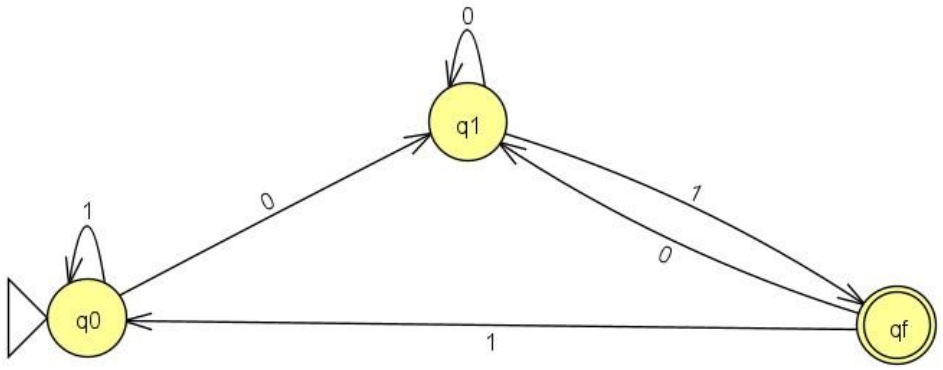
ii. The DFA that accepts all the strings that end with 01.

Q = { q0, q1, qf}
start state = q0,
Final state = qf

- Transition table

State	Inputs	
	0	1
q0	q1	q1
q1	q1	qf
qf	q1	q0

- Transition Diagram



SOURCE CODE:

```
#include<stdio.h>

enum states { q0, q1, qf};

int main()
{
    char input[20];
    enum states curr_state = q0;
    int i = 0;

    printf("\nEnter a binary string: ");
    gets(input);  // Note: gets() is unsafe; consider using fgets() instead in real-world code
```

```
    char ch = input[i];
    while(ch != '\0')
    {
        switch(curr_state)
        {
            case q0:
                if(ch == '0')
                    curr_state = q1;
                else
                    curr_state = q0;
                break;

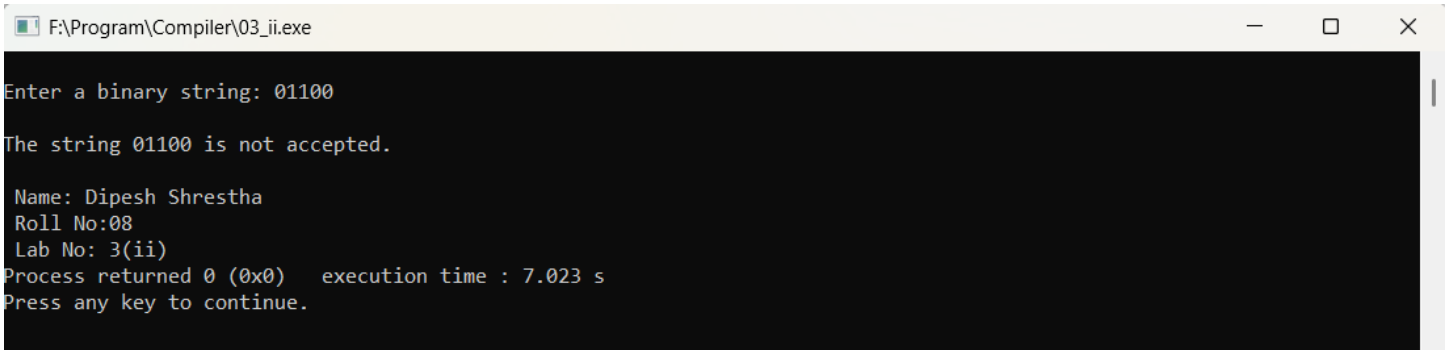
            case q1:
                if(ch == '1')
                    curr_state = qf;
                else
                    curr_state = q1;
                break;

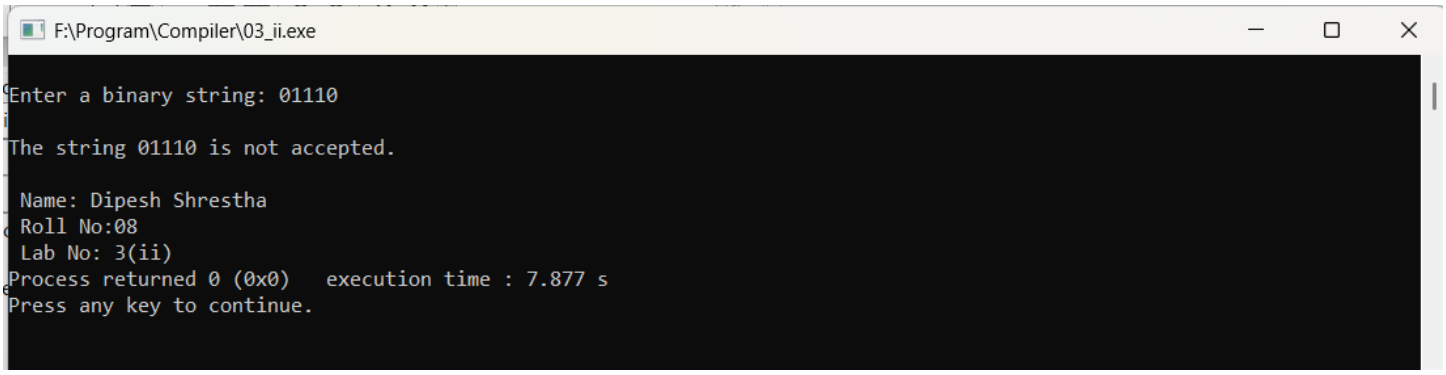
            case qf:
                if(ch == '0')
                    curr_state = q1;
                else
                    curr_state = q0;
                break;
        }
        ch = input[++i];
    }

    if(curr_state == qf)
        printf("\nThe string %s is accepted.", input);
    else
        printf("\nThe string %s is not accepted.", input);
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 3(ii)");


    return 0;
}
```

OUTPUT:


F:\Program\Compiler\03_ii.exe
Enter a binary string: 01100
The string 01100 is not accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(ii)
Process returned 0 (0x0) execution time : 7.023 s
Press any key to continue.


F:\Program\Compiler\03_ii.exe
Enter a binary string: 01110
The string 01110 is not accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(ii)
Process returned 0 (0x0) execution time : 7.877 s
Press any key to continue.

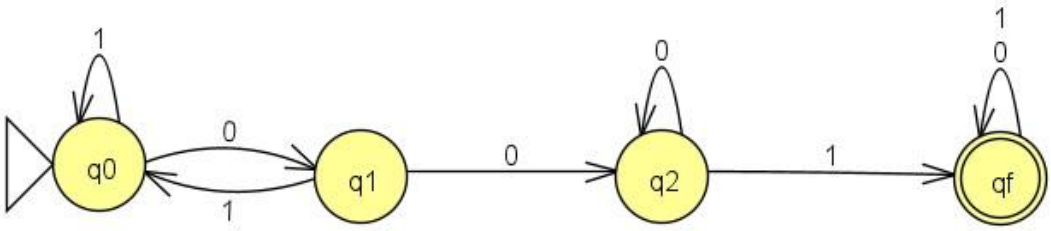
iii. The DFA that accepts all string that contains substring 001.

Q = {q0, q1, q2, qf}
start state = q0,
Final state = qf

• Transition Table

State	Inputs	
	0	1
q0	q1	q0
q1	q2	q0
q2	q2	qf
qf	qf	qf

• Transition Diagram



SOURCE CODE:

```
#include<stdio.h>

enum states { q0, q1, q2, qf };

int main()
{
    enum states curr_state = q0;
    char string[20], ch;
    int i = 0;

    printf("\nEnter a string: ");
    gets(string);  // Note: `gets()` is unsafe. Consider using `fgets()` in real-world code.

    ch = string[i];
    while(ch != '\0')
    {
        switch(curr_state)
```

```
        {
            case q0:
                if(ch == '0')
                    curr_state = q1;
                else
                    curr_state = q0;
                break;

            case q1:
                if(ch == '0')
                    curr_state = q2;
                else
                    curr_state = q0;
                break;

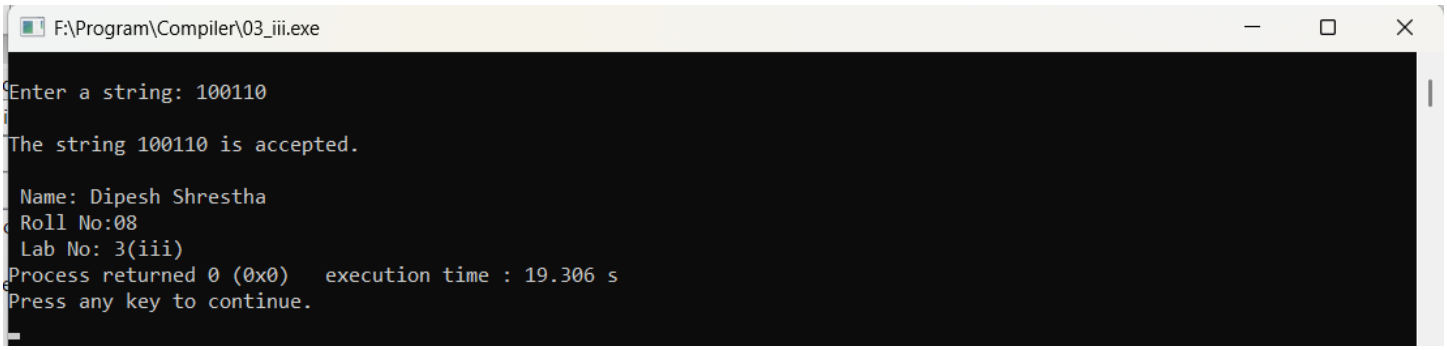
            case q2:
                if(ch == '0')
                    curr_state = q2;
                else
                    curr_state = qf;
                break;

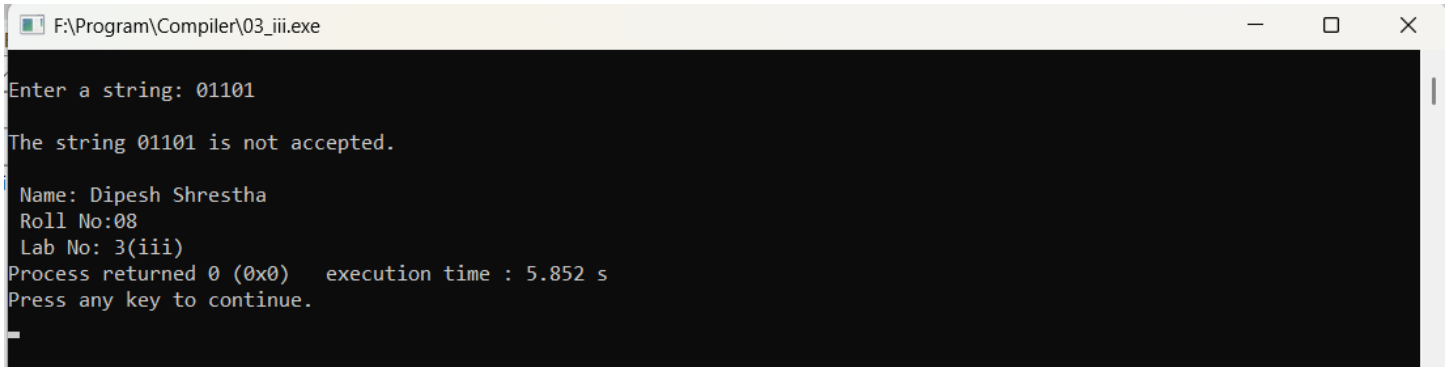
            case qf:
                if(ch == '0' || ch == '1')
                    curr_state = qf;
                break;
        }
        ch = string[++i];
    }

    if(curr_state == qf)
        printf("\nThe string %s is accepted.", string);
    else
        printf("\nThe string %s is not accepted.", string);
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 3(iii)");

    return 0;
}
```

OUTPUT:


F:\Program\Compiler\03_iii.exe
Enter a string: 100110
The string 100110 is accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(iii)
Process returned 0 (0x0) execution time : 19.306 s
Press any key to continue.


F:\Program\Compiler\03_iii.exe
Enter a string: 01101
The string 01101 is not accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 3(iii)
Process returned 0 (0x0) execution time : 5.852 s
Press any key to continue.

LAB: 4

TITLE: IMPLEMENTATION OF REGULAR EXPRESSION

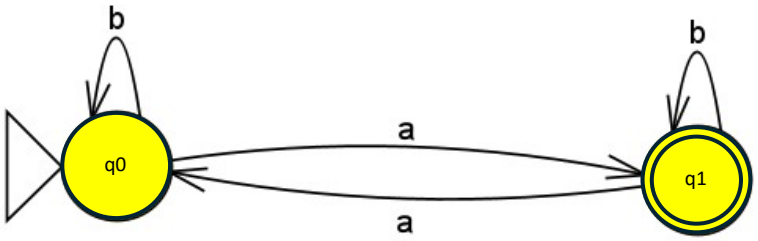
Objective: Write a C program to implement following DFA for L={ Set of all strings over ∑ = {a , b } such that string has odd no. of a's

Regular Expression: b*a(b*ab*a)*b*.

- Transition Table

State	Inputs	
	a	b
q0	q1	q0
q1	q0	q1

- Transition Diagram



SOURCE CODE:

```
#include<stdio.h>

enum states { q0, q1 };

int main()
{
    enum states curr_state = q0;
    char input[20], ch;
    int i = 0;
```

```
    printf("\nEnter a string: ");
    gets(input);

    ch = input[i];
    while (ch != '\0')
    {
        switch (curr_state)
        {
            case q0:
                if (ch == 'a')
                    curr_state = q1;
                else
                    curr_state = q0;
                break;

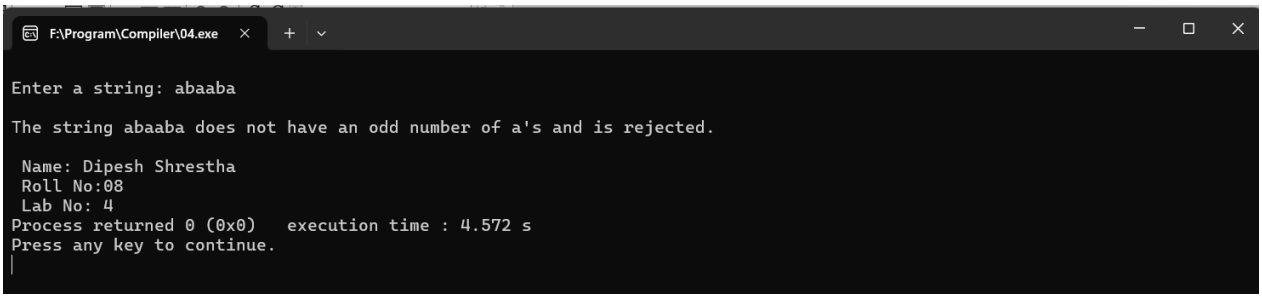
            case q1:
                if (ch == 'a')
                    curr_state = q0;
                else
                    curr_state = q1;
                break;
        }
        ch = input[++i];
    }

    if (curr_state == q1)
        printf("\nThe string %s has an odd number of a's and is accepted.", input);
    else
        printf("\nThe string %s does not have an odd number of a's and is rejected.", input);
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 4");

    return 0;
}
```

OUTPUT:


F:\Program\Compiler\04.exe
Enter a string: abaab
The string abaab has an odd number of a's and is accepted.
Name: Dipesh Shrestha
Roll No:08
Lab No: 4
Process returned 0 (0x0)   execution time : 5.814 s
Press any key to continue.


F:\Program\Compiler\04.exe
Enter a string: abaaba
The string abaaba does not have an odd number of a's and is rejected.
Name: Dipesh Shrestha
Roll No:08
Lab No: 4
Process returned 0 (0x0)   execution time : 4.572 s
Press any key to continue.

LAB: 5

TITLE: COMMENT IN C.

Objective: Write a C program to identify weather a given line is a comment.

ALGORITHM

1. Read the input string.
2. If the string is starting '/' and check next character is '/'or '*'
3. If condition satisfies print comment.
4. Else not a comment.

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>

enum States
{
    S0, // Start state
    S1, // After first '/'
    S2, // Single-line comment
    S3, // Multi-line comment
    S4, // After '*' in multi-line comment
    S5, // End of multi-line comment
    S6  // Invalid state
};

enum States dfa_comment_checker(char input_string[])
{
    enum States state = S0; // Initial state
    int i = 0;
    char ch;

    while ((ch = input_string[i]) != '\0')
    {
        switch (state)
```

```
{
case S0:
    if (ch == '/')
        state = S1;
    else
        state = S6;
    break;

case S1:
    if (ch == '/')
        state = S2;
    else if (ch == '*')
        state = S3;
    else
        state = S6;
    break;

case S2:
    state = S2;
    break;

case S3:
    if (ch == '*')
        state = S4;
    else
        state = S3;
    break;

case S4:
    if (ch == '/')
        state = S5;
    else if (ch == '*')
        state = S4;
    else
        state = S3;
    break;

case S5:
    state= S5;
    break;
```

```
            }

            i++;
        }

        return state;
    }

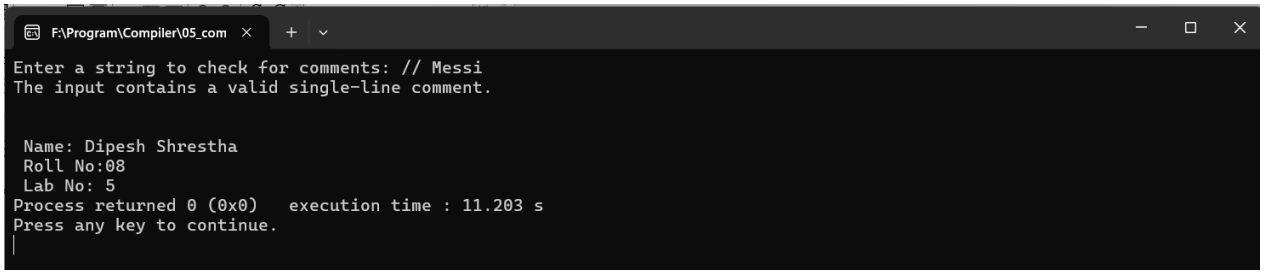
    int main()
    {
        char input_string[100];

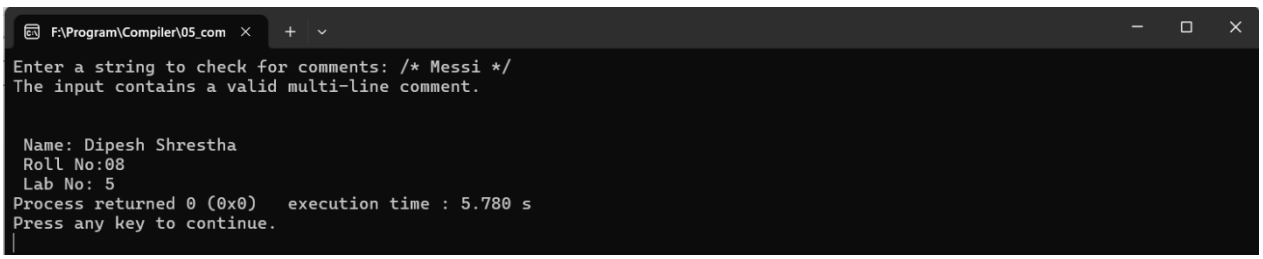
        printf("Enter a string to check for comments: ");
         gets(input_string);

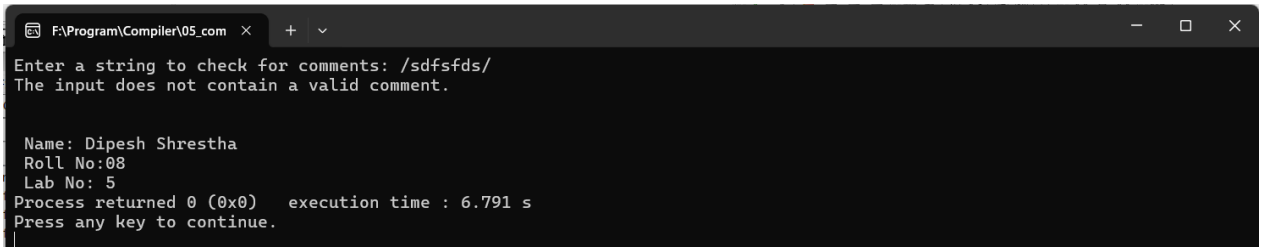
        enum States final_state = dfa_comment_checker(input_string);

        if (final_state == S2)
            printf("The input contains a valid single-line comment.\n");
        else if (final_state == S5)
            printf("The input contains a valid multi-line comment.\n");
        else
            printf("The input does not contain a valid comment.\n");
        printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 5");
        return 0;
    }
```

OUTPUT:






F:\Program\Compiler\05_com
Enter a string to check for comments: /sdfsfds/
The input does not contain a valid comment.
Name: Dipesh Shrestha
Roll No:08
Lab No: 5
Process returned 0 (0x0) execution time : 6.791 s
Press any key to continue.

LAB: 6

TITLE: COMPUTATION OF FIRST

Objective: Write a C program to illustrate computation of FIRST.

SOURCE CODE

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
void FIRST(char[], char);
void addToResultSet(char[], char);
int numOfProductions;
char productionSet[10][10];

int main() {
    int i;
    char choice;
    char ch;
    char result[20];

    printf("How many number of productions?: ");
    scanf("%d", &numOfProductions);

    printf("Enter the Production in the form V=(VUT)*\n ");

    for (i = 0; i < numOfProductions; i++) {
        printf("Enter productions number %d: ", i + 1);
        scanf("%s", productionSet[i]);
    }

    do {
        printf("Find the First of: ");
        scanf(" %c", &ch);
        result[0] = '\0';
        FIRST(result, ch);
        printf("\nFIRST(%c) = {", ch);
        for (i = 0; result[i] != '\0'; i++) {
            printf("%c", result[i]);
            if (result[i + 1] != '\0') {
                printf(", ");
            }
        }
```

```
        printf("}\n");
        printf("Press 'y' to continue: ");
        scanf(" %c", &choice);
    } while (choice == 'y' || choice == 'Y');

    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 5");
    return 0;
}

void FIRST(char* Result, char ch) {
    int i, j, k;
    char subResult[20];
    int foundEpsilon;
    subResult[0] = '\0';

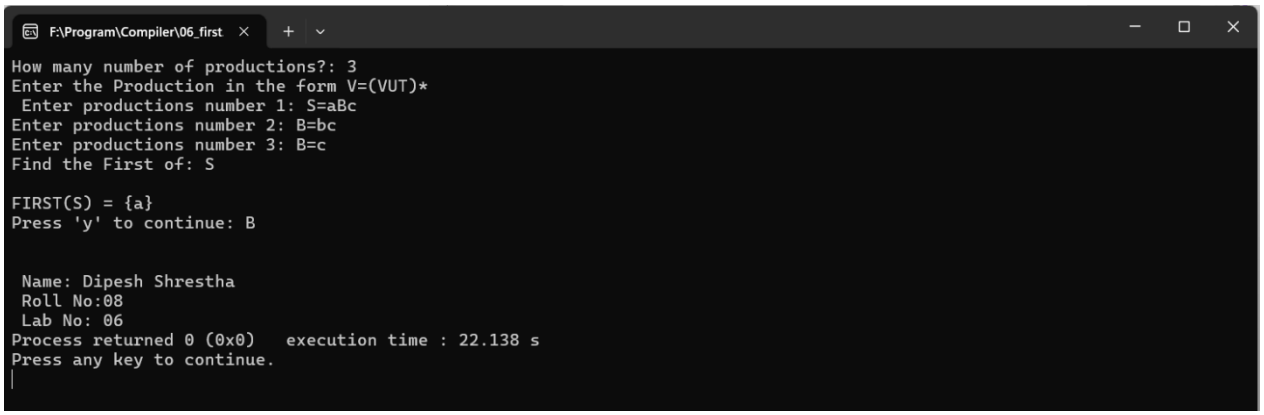
    if (!isupper(ch)) {
        addToResultSet(Result, ch);
        return;
    }

    for (i = 0; i < numOfProductions; i++) {
        if (productionSet[i][0] == ch) {
            if (productionSet[i][2] == '$') {
                addToResultSet(Result, '$');
            } else {
                j = 2;
                while (productionSet[i][j] != '\0') {
                    foundEpsilon = 0;
                    FIRST(subResult, productionSet[i][j]);
                    for (k = 0; subResult[k] != '\0'; k++) {
                        addToResultSet(Result, subResult[k]);
                    }
                    for (k = 0; subResult[k] != '\0'; k++) {
                        if (subResult[k] == '$') {
                            foundEpsilon = 1;
                            break;
                        }
                    }
                    if (!foundEpsilon) {
                        break;
                    }
                    j++;
                }
```

```
        }
      }
    }
}

void addToResultSet(char Result[], char val) {
    int k;
    for (k = 0; Result[k] != '\0'; k++) {
        if (Result[k] == val) {
            return;
        }
    }
    Result[k] = val;
    Result[k + 1] = '\0';
}
```

OUTPUT



LAB: 7

TITLE: COMPUTATION OF FOLLOW

OBJECTIVE: Write a C program to illustrate computation of FOLLOW.

SOURCE CODE:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

void followfirst(char, int, int);
void follow(char c);
void findfirst(char, int, int);

int count, n = 0;
char calc_first[10][100];
char calc_follow[10][100];
int m = 0;
char production[10][10];
char f[10], first[10];
int k;
char ck;
int e;

int main()
{
    int jm = 0;
```

```
int km = 0;
int i, choice;
char c, ch;
printf("Enter the number of productions: ");
scanf("%d", &count);

printf("Enter the productions (e.g., E=TR| #=EPSILON):\n");
for (i = 0; i < count; i++)
{
    scanf("%s", production[i]);
}

int key;
char done[count];
int ptr = -1;

for(k = 0; k < count; k++)
{
    for(key = 0; key < 100; key++)
    {
        calc_first[k][key] = '!';
    }
}

int point1 = 0, point2, x;
```

```
for(k = 0; k < count; k++)
{
    c = production[k][0];
    point2 = 0;
    x = 0;

    for(key = 0; key <= ptr; key++)
        if(c == done[key])
            x = 1;

    if (x == 1)
        continue;

    findfirst(c, 0, 0);
    ptr += 1;

    done[ptr] = c;
    printf("\n First(%c) = { ", c);
    calc_first[point1][point2++] = c;

    for(i = 0 + jm; i < n; i++)
    {
        int lark = 0, chk = 0;

        for(lark = 0; lark < point2; lark++)
        {
```

```
            if (first[i] == calc_first[point1][lark])
            {
                chk = 1;
                break;
            }
        }

        if(chk == 0)
        {
            printf("%c, ", first[i]);
            calc_first[point1][point2++] = first[i];
        }
    }

    printf("}\n");
    jm = n;
    point1++;
}

printf("\n");
printf("----------------------------------------------\n\n");

char donee[count];
ptr = -1;

for(k = 0; k < count; k++)
```

```
{
    for(key = 0; key < 100; key++)
    {
        calc_follow[k][key] = '!';
    }
}

point1 = 0;
int land = 0;

for(e = 0; e < count; e++)
{
    ck = production[e][0];
    point2 = 0;
    x = 0;

    for(key = 0; key <= ptr; key++)
        if(ck == donee[key])
            x = 1;

    if (x == 1)
        continue;

    land += 1;
    follow(ck);
    ptr += 1;
```

```
donee[ptr] = ck;
printf(" Follow(%c) = { ", ck);
calc_follow[point1][point2++] = ck;

for(i = 0 + km; i < m; i++)
{
    int lark = 0, chk = 0;

    for(lark = 0; lark < point2; lark++)
    {
        if (f[i] == calc_follow[point1][lark])
        {
            chk = 1;
            break;
        }
    }

    if(chk == 0)
    {
        printf("%c, ", f[i]);
        calc_follow[point1][point2++] = f[i];
    }
}

printf(" }\n\n");
```

```
        km = m;
        point1++;
    }

    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 5");
}

void follow(char c)
{
    int i, j;

    if(production[0][0] == c)
    {
        f[m++] = '$';
    }

    for(i = 0; i < 10; i++)
    {
        for(j = 2; j < 10; j++)
        {
            if(production[i][j] == c)
            {
                if(production[i][j+1] != '\0')
                {
                    followfirst(production[i][j+1], i, (j+2));
                }
```

```
            if(production[i][j+1]=='\0' && c!=production[i][0])
            {
                follow(production[i][0]);
            }
        }
    }
}

void findfirst(char c, int q1, int q2)
{
    int j;

    if(!(isupper(c)))
    {
        first[n++] = c;
    }

    for(j = 0; j < count; j++)
    {
        if(production[j][0] == c)
        {
            if(production[j][2] == '#')
            {
                if(production[q1][q2] == '\0')
```

```
                first[n++] = '#';
            else if(production[q1][q2] != '\0' && (q1 != 0 || q2 != 0))
            {
                findfirst(production[q1][q2], q1, (q2+1));
            }
            else
                first[n++] = '#';
        }
        else if(!isupper(production[j][2]))
        {
            first[n++] = production[j][2];
        }
        else
        {
            findfirst(production[j][2], j, 3);
        }
    }
  }
}

void followfirst(char c, int c1, int c2)
{
    int k;

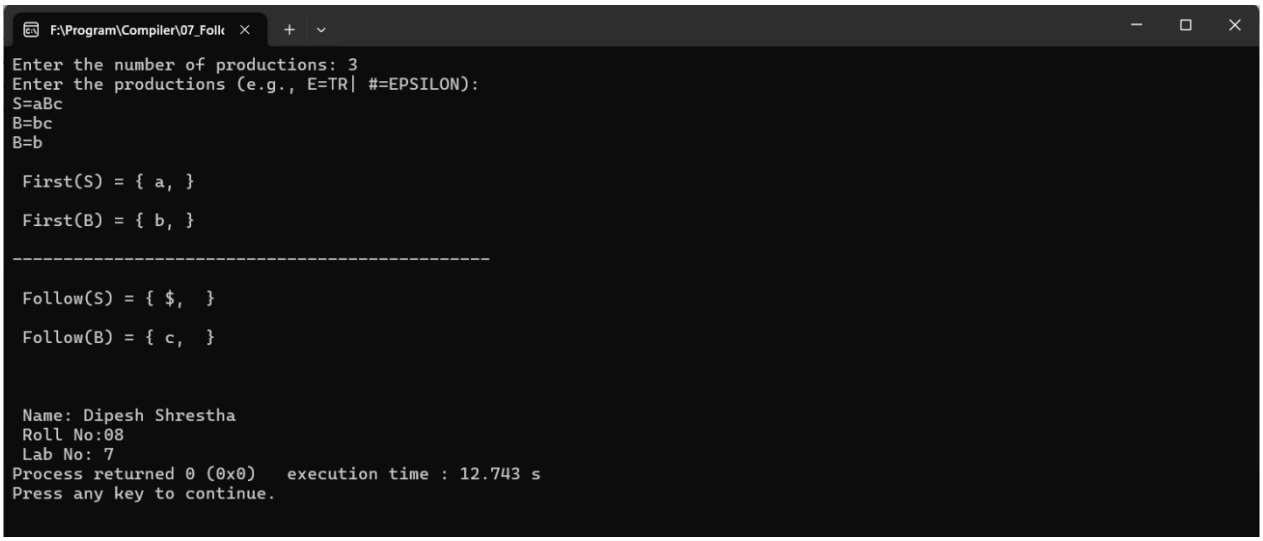
    if(!(isupper(c)))
        f[m++] = c;
```

```
else
{
    int i = 0, j = 1;
    for(i = 0; i < count; i++)
    {
        if(calc_first[i][0] == c)
            break;
    }

    while(calc_first[i][j] != '!')
    {
        if(calc_first[i][j] != '#')
        {
            f[m++] = calc_first[i][j];
        }
        else
        {
            if(production[c1][c2] == '\0')
            {
                follow(production[c1][0]);
            }
            else
            {
                followfirst(production[c1][c2], c1, c2+1);
            }
        }
```

```
        j++;
      }
    }
}
```

OUTPUT:


F:\Program\Compiler\07_Follc
Enter the number of productions: 3
Enter the productions (e.g., E=TR| #=EPSILON):
S=aBc
B=bc
B=b
First(S) = { a, }
First(B) = { b, }
Follow(S) = { $, }
Follow(B) = { c, }
Name: Dipesh Shrestha
Roll No:08
Lab No: 7
Process returned 0 (0x0) execution time : 12.743 s
Press any key to continue.

LAB: 8

TITLE: IDENTIFIER VALIDATION

OBJECTIVE: Write a C program to test a given identifier is valid or not. .

ALGORITHM/ LOGIC:

1. Read the input string.
2. Check the start character of the string is numerical or any special character except '_' then print it is not a valid identifier.
3. Otherwise: print it is valid identifier if remaining character of string does't contain any special character except '_'.

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isValidIdentifier(char str[]) {
    if (!isalpha(str[0]) && str[0] != '_') {
        return 0;
    }

    for (int i = 1; str[i] != '\0'; i++) {
        if (!isalnum(str[i]) && str[i] != '_') {
            return 0;
        }
    }
```

```
    return 1;
}

int main() {
    char str[50];

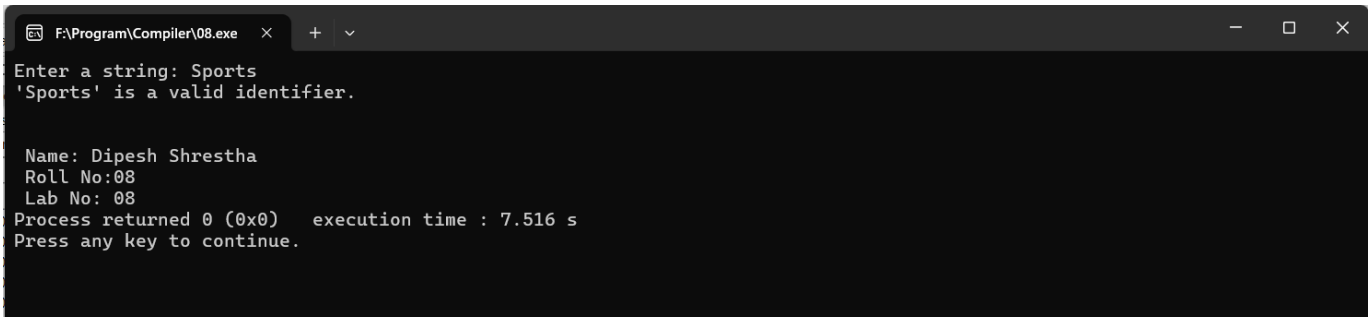
    printf("Enter a string: ");
    scanf("%s", str);

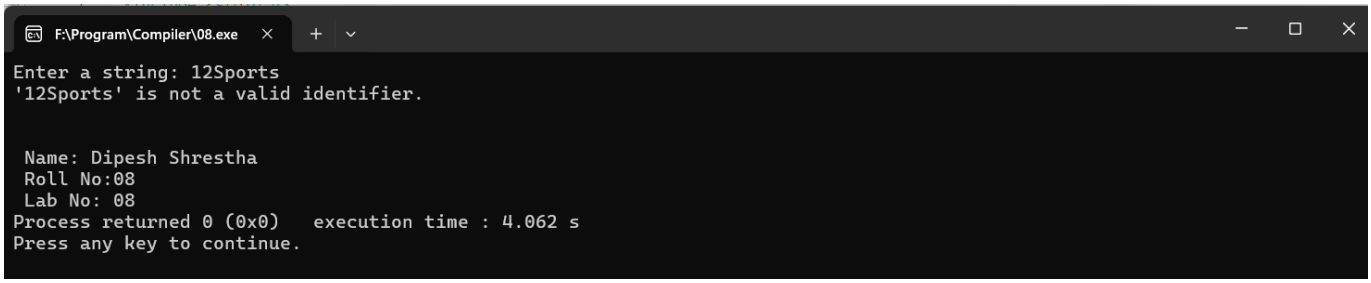
    if (isValidIdentifier(str)) {
        printf("'%s' is a valid identifier.\n", str);
    } else {
        printf("'%s' is not a valid identifier.\n", str);
    }

    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 02");

    return 0;
}
```

OUTPUT:


F:\Program\Compiler\08.exe
Enter a string: Sports
'Sports' is a valid identifier.
Name: Dipesh Shrestha
Roll No:08
Lab No: 08
Process returned 0 (0x0) execution time : 7.516 s
Press any key to continue.


F:\Program\Compiler\08.exe
Enter a string: 12Sports
'12Sports' is not a valid identifier.
Name: Dipesh Shrestha
Roll No:08
Lab No: 08
Process returned 0 (0x0) execution time : 4.062 s
Press any key to continue.

LAB: 9

TITLE: LEXICAL ANALYSIS

OBJECTIVE: Write a C program to simulate lexical analysis for validating operators.

SOURCE CODE

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[3];
    int valid = 0;

    printf("Enter value to be identified: ");
    scanf("%s", str);

    if (((str[0] == '&' || str[0] == '|') && str[0] == str[1]) || (str[0] == '!' && str[1] == '\0')) {
        printf("\nIt is a Logical operator");
        valid = 1;
    }

    char relational[4] = {'<', '>', '!', '='};
    for (int i = 0; i < 4; i++) {
        if (str[0] == relational[i] && (str[1] == '=' || str[1] == '\0')) {
            printf("\nIt is a Relational Operator");
            valid = 1;
            break;
```

```
    }
}

char bitwise[4] = {'&', '^', '~', '|'};
for (int i = 0; i < 4; i++) {
    if ((str[0] == bitwise[i] && str[1] == '\0') || ((str[0] == '<' || str[0] == '>') && str[1] == str[0])) {
        printf("\nIt is a Bitwise Operator");
        valid = 1;
        break;
    }
}

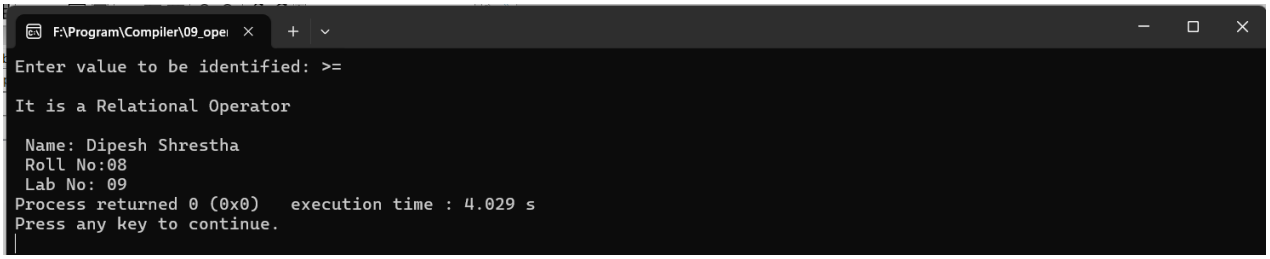
if (str[0] == '?' && str[1] == ':') {
    printf("\nIt is a Ternary operator");
    valid = 1;
}

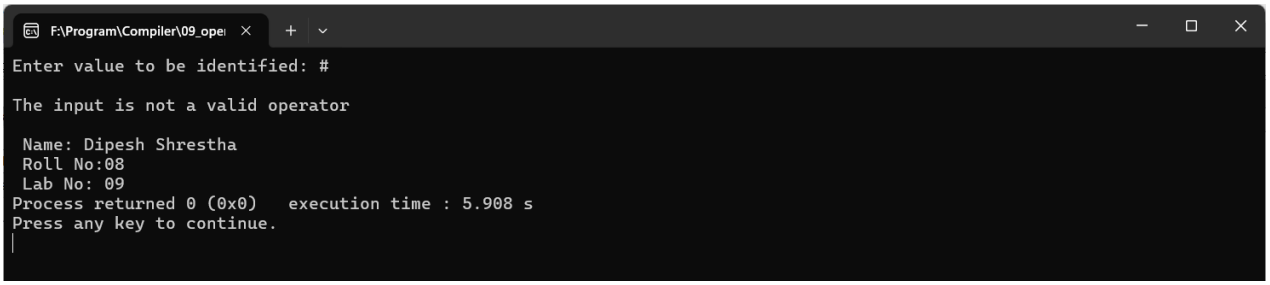
char arithmetic[5] = {'+', '-', '*', '/', '%'};
for (int i = 0; i < 5; i++) {
    if ((str[0] == '+' || str[0] == '-') && str[0] == str[1]) {
        printf("\nIt is a Unary operator");
        valid = 1;
        break;
    } else if ((str[0] == arithmetic[i] && str[1] == '=') || (str[0] == '=' && str[1] == ' ')) {
        printf("\nIt is an Assignment operator");
```

```
            valid = 1;
            break;
        } else if (str[0] == arithmetic[i] && str[1] == '\0') {
            printf("\nIt is an Arithmetic operator");
            valid = 1;
            break;
        }
    }

    if (!valid) {
        printf("\nThe input is not a valid operator");
    }
printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 09");
    return 0;
}
```

OUTPUT:





LAB: 10

TITLE: TYPE CHECKING

OBJECTIVE: Write a C program to implement a symbol table to check type.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int main() {
    int i = 0, j = 0, x = 0, n, flag = 0;
    void *p, *add[15];
    char srch, b[15], d[15], c;

    printf("Enter expression terminated by $: ");
    while((c = getchar()) != '$') {
        b[i] = c;
        i++;
    }
    b[i] = '\0'; // Null-terminate the string
    n = i;

    printf("Given expression is: %s\n", b);
    printf("Symbol table:\n");
    printf("Symbol\t Address\t\t Type\n");

    for (j = 0; j < n; j++) {
        c = b[j];
        if (isalpha(c)) {
            if (j == n - 1 || b[j + 1] == '+' || b[j + 1] == '-' || b[j + 1] == '*' || b[j + 1] == '=') {
                p = malloc(sizeof(char));
                add[x] = p;
                d[x] = c;
                printf("%c\t %p\t identifier\n", c, p);
                x++;
            }
        }
    }
```

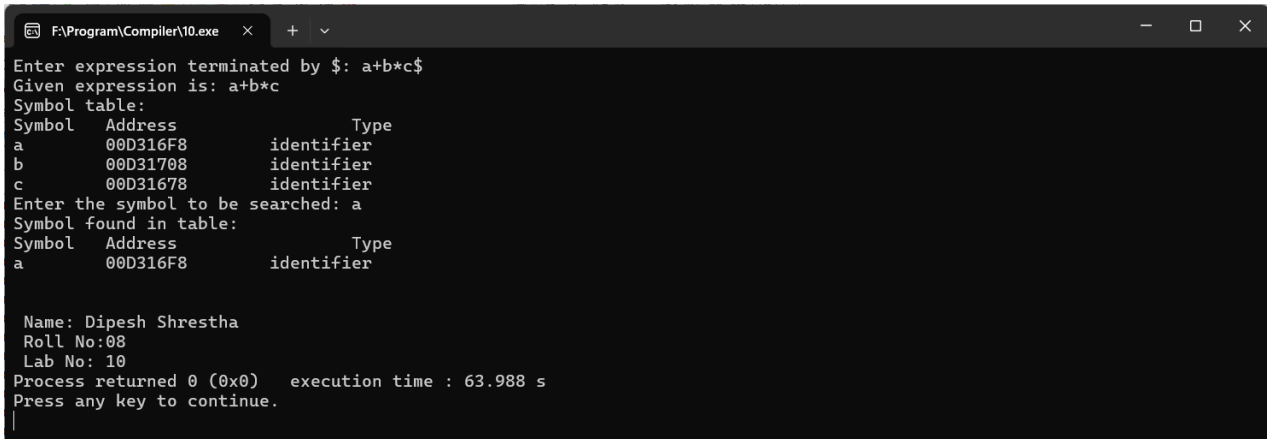
```
    printf("Enter the symbol to be searched: ");
    scanf(" %c", &srch);

    for (i = 0; i < x; i++) {
        if (srch == d[i]) {
            printf("Symbol found in table:\n");
            printf("Symbol\t Address\t\t Type\n");
            printf("%c\t %p\t identifier\n", srch, add[i]);
            flag = 1;
            break;
        }
    }

    if (flag == 0)
        printf("Symbol not found in the table.\n");

    for (i = 0; i < x; i++) {
        free(add[i]);
    }
printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 10");
    return 0;
}
```

OUTPUT:



LAB: 11

TITLE: SHIFT REDUCE PARSER

OBJECTIVE: Write a C program to implement Shift Reduce Parser.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
char ip_sym[20], stack[20];
int ip_ptr = 0, st_ptr = 0, len, i;
char temp[2], temp2[2];
char act[20];
void check();
void main() {
    printf("\n\t\t Shift Reduce Parser\n");
    printf("\n Grammar\n");
    printf("\n E->E+E \n E->E/E");
    printf("\n E->E*E \n E->a/b");

    printf("\n Enter the input symbol:");
    gets(ip_sym);

    printf("\n\t Stack implementation table");
    printf("\n Stack \t\t input symbol\t\taction");
    printf("\n----------\t\t----------\t\t---------\n");
    printf("\n$\t\t%s$\t\t\t---", ip_sym);
```

```
    strcpy(act, "Shift ");
    temp[0] = ip_sym[ip_ptr];
    temp[1] = '\0';
    strcat(act, temp);
    len = strlen(ip_sym);
    for (i = 0; i < len; i++) {
        stack[st_ptr] = ip_sym[ip_ptr];
        stack[st_ptr + 1] = '\0';
        ip_sym[ip_ptr] = ' ';
        ip_ptr++;
        printf("\n $%s\t\t%s$\t\t\t%s", stack, ip_sym, act);
        strcpy(act, "Shift ");
        temp[0] = ip_sym[ip_ptr];
        temp[1] = '\0';
        strcat(act, temp);
        check();
        st_ptr++;
    }
    st_ptr++;
    check();
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 11");

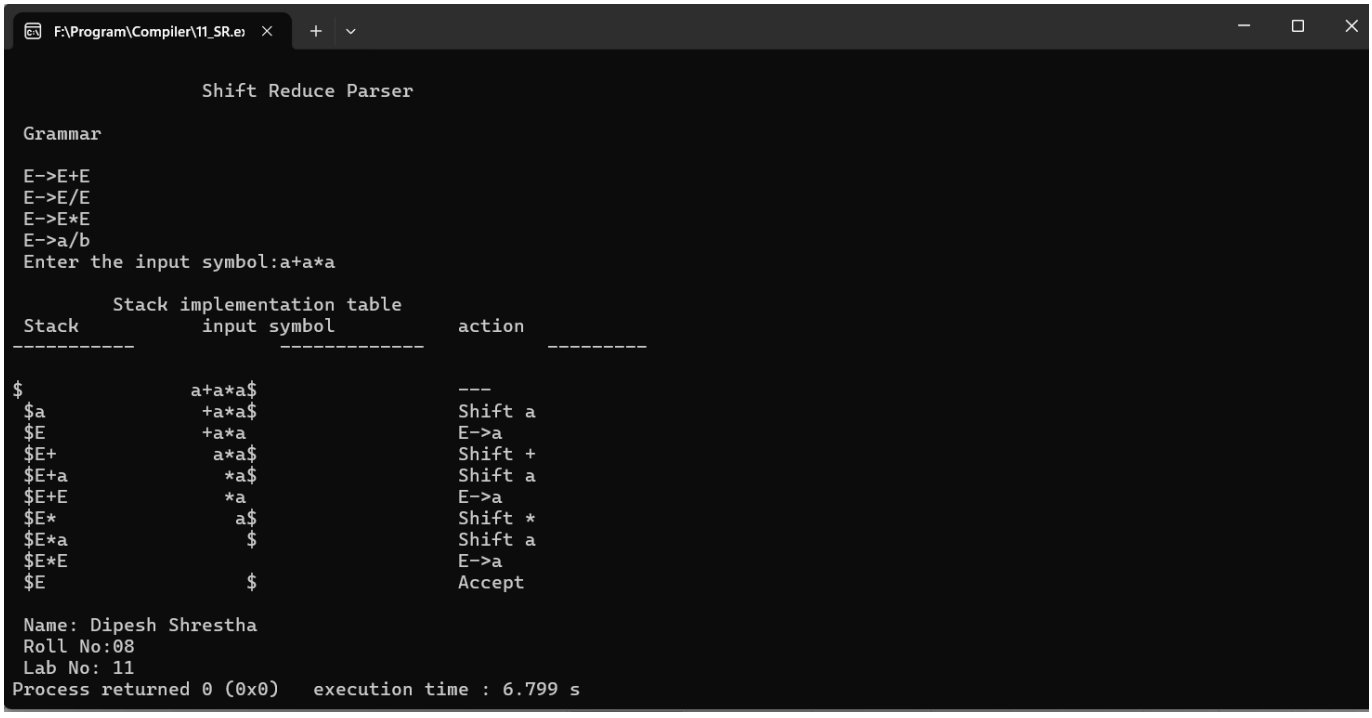
}

void check() {
```

```
int flag = 0;
temp2[0] = stack[st_ptr];
temp2[1] = '\0';
if ((!strcmp(temp2, "a")) || (!strcmp(temp2, "b"))) {
    stack[st_ptr] = 'E';
    if (!strcmp(temp2, "a"))
        printf("\n $%s\t\t%s\t\t\tE->a", stack, ip_sym);
    else
        printf("\n $%s\t\t%s$\t\t\tE->b", stack, ip_sym);
    flag = 1;
}
if ((!strcmp(temp2, "+")) || (!strcmp(temp2, "*")) || (!strcmp(temp2, "/"))) {
    flag = 1;
}
if ((!strcmp(stack, "E+E")) || (!strcmp(stack, "E/E")) || (!strcmp(stack, "E*E"))) {
    strcpy(stack, "E");
    st_ptr = 0;
    if (!strcmp(stack, "E+E"))
        printf("\n $%s\t\t%s\t\t\tE->E+E", stack, ip_sym);
    else if (!strcmp(stack, "E/E"))
        printf("\n $%s\t\t%s\t\t\tE->E/E", stack, ip_sym);
    else if (!strcmp(stack, "E*E"))
        printf("\n $%s\t\t%s\t\t\tE->E*E", stack, ip_sym);
    flag = 1;
}
```

```
    if (!strcmp(stack, "E") && ip_ptr == len) {
        printf("\n $%s\t\t%s$\t\t\tAccept", stack, ip_sym);
          printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 11");
        exit(0);
    }
    if (flag == 0) {
        printf("\n%s\t\t\t%s\t\t Reject", stack, ip_sym);
    }
    return;
}
```

OUTPUT:



LAB: 12

TITLE: IMPLEMENTATION OF SYMBOL TABLE

OBJECTIVE: Write a C program to implement Symbol Table.

LOGIC/ALGORITHM

1. start the program for performing insert, display, delete, search and modify option in symbol table
2. Define the structure of the symbol table.
3. Enter the choice for performing the operation in the symbol table.
4. If the entered choice is 1, search the symbol table for the symbol to inserted. if the symbol table is already present, it displays "Duplicate symbol". Else, insert the symbol table and the corresponding address in the symbol table.
5. If the entered choice is 2, the symbol present in the symbol table are displayed.
6. if the entered choice is 3, the symbol to be deleted is searched in the symbol table.
7. if it is not found in the symbol table it displays "Label not found". Else the symbol is deleted.
8. if the entered choice is 5, the symbol to be modified is searched in the symbol table. the label or address or both can be modified.

SOURCE CODE

```
#include<stdio.h>

#include<string.h>

#define null 0

int size = 0;

void insert();

void del();

int search(char lab[]);

void modify();

void display();

struct symbtab
```

```
{
    char label[10];
    int addr;
    struct symtab *next;
};
struct symbtab *first, *last;
void main()
{
    int op;
    int y;
    char la[10];
    //clrscr();
    do
    {
        printf("\n symbol table implementation\n");
        printf("1.Insert\n");
        printf("2. Display\n");
        printf("3. Delete\n");
        printf("4. Search\n");
        printf("5. Modify\n");
        printf("6. End\n");
        printf("Enter your option :");
        scanf("%d",&op);
        switch(op)
        {
```

```
case 1:
    insert();
    display();
    break;
case 2:
    display();
    break;
case 3:
    del();
    display();
    break;
case 4:
    printf("Enter the label to be searched");
    scanf("%s",la);
    y=search(la);
    if(y==1)
    {
        printf("The label is already in the symbol table");
    }
    else
    {
        printf("The label is not found in the symbol table");
    }
    break;
case 5:
```

```
            modify();
            display();
            break;
        case 6:
            printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 12");
        }
    }
    while(op < 6);
    //getch();
}
void insert()
{
    int n;
    char l[10];
    printf("Enter the label:");
    scanf("%s",l);
    n=search(l);
    if(n==1)
    {
        printf("The label already exits.Duplicate can't be inserted.");
    }
    else
    {
        struct symbtab *p;
        p=malloc(sizeof(struct symbtab));
```

```
        strcpy(p->label,l);
        printf("Enter the address :");
        scanf("%d",&p->addr);
        p->next=null;
        if(size==0 )
        {
           first = p;
           last =p;
        }
        else
        {
           last->next=p;
           last=p;
        }
        size++;
    }
}
void display()
{
    int i;
    struct symbtab *p;
    p=first;
    printf("Label\t Address\n");
    for(i=0; i<size; i++)
    {
```

```
        printf("%s\t%d\n",p->label,p->addr);
        p=p->next;
    }
}
int search(char lab[])
{
    int i, flag=0;
    struct symbtab *p;
    p=first;
    for(i=0; i<size; i++)
    {
        if(strcmp(p->label,lab)==0)
        {
            flag = 1;
        }
        p=p->next;
    }
    return flag;
}
void modify()
{
    char l[10], nl[10];
    int add, choice, i, s;
    struct symbtab *p;
    p=first;
```

```
printf("What do you want to modify?");
printf("1. Only the label\n");
printf("2. Only the address of a particular label");
printf("3. Both the label and address\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
case 1:
    printf("Enter the old label\n");
    scanf("%s",l);
    printf("Enter the new label\n");
    scanf("%s",nl);
    s=search(l);
    if(s==0)
    {
        printf("No such label");
    }
    else
    {
        for(i=0; i<size; i++)
        {
            if(strcmp(p->label,l)==0)
            {
                strcpy(p->label,nl);
```

```
            }
            p=p->next;
        }
    }
    break;
case 2:
    printf("Enter the label whose address is to be modified\n");
    scanf("%s",l);
    printf("Enter the new address\n");
    scanf("%d",&add);
    s=search(l);
    if(s==0)
    {
        printf("No such label");
    }
    else
    {
        for(i=0; i<size; i++)
        {
            if(strcmp(p->label,l)==0)
            {
                p->addr=add;
            }
            p=p->next;
        }
```

```
    }
    break;

case 3:
    printf("Enter the old label:");
    scanf("%s",l);
    printf("Enter the new label");
    scanf("%s",nl);
    printf("Enter the new address :");
    scanf("%d",&add);
    s=search(l);
    if(s==0)
    {
        printf("No such label");
    }
    else
    {
        for(i=0; i<size; i++)
        {
            if(strcmp(p->label,l)==0)
            {
                strcpy(p->label,nl);
                p->addr=add;
            }
            p=p->next;
```

```
            }
        }
        break;
    }
    printf("\n\n Name: Dipesh Shrestha\n Roll No:08 \n Lab No: 12");
}
void del()
{
    int a;
    char l[10];
    struct symbtab *p, *q;
    p=first;
    printf("Enter the label to be deleted\n");
    scanf("%s",l);
    a = search(l);
    if(a==0)
    {
        printf("Label not found\n");
    }
    else
    {
        if(strcmp(first->label,l)==0)
        {
            first = first->next;
        }
```

```
    else if(strcmp(last->label,l)==0)
    {
        q = p->next;
        while(strcmp(q->label,l)!=0)
        {
            p= p->next;
            q= q->next;
        }
        p->next=null;
        last=p;
    }

    else
    {
        q=p->next;
        while(strcmp(q->label,l)!=0)
        {
            p =p->next;
            q =q->next;
        }
        p->next = q->next;
    }
    size--;
}
```

}

OUTPUT:

- **Insert**



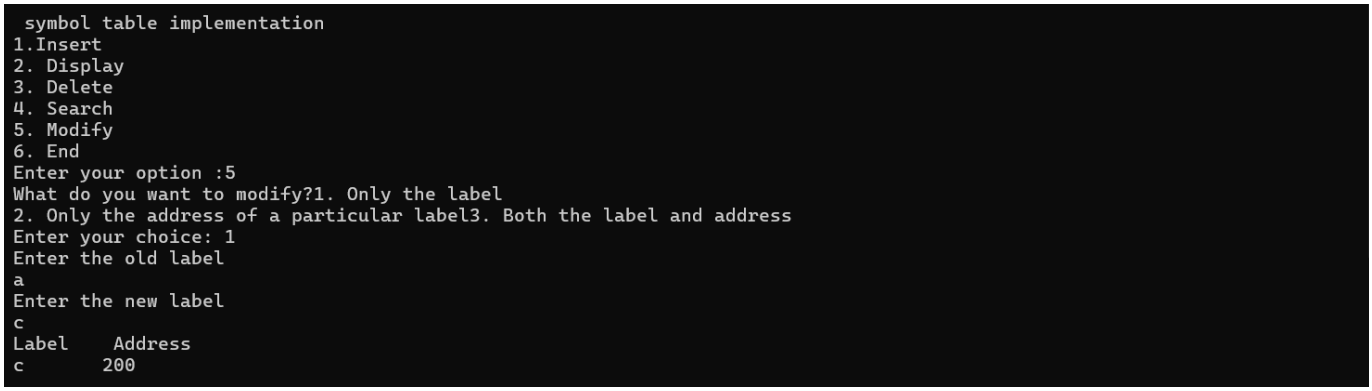
- **Display**



- **Search**



- **Modify**



- **Delete**


symbol table implementation
1.Insert
2. Display
3. Delete
4. Search
5. Modify
6. End
Enter your option :3
Enter the label to be deleted
c
Label Address
symbol table implementation
1.Insert
2. Display
3. Delete
4. Search
5. Modify
6. End
Enter your option :6
Name: Dipesh Shrestha
Roll No:08
Lab No: 12
Process returned 6 (0x6) execution time : 273.157 s
Press any key to continue.

LAB: 13

TITLE: IMPLEMENTATION OF SLR(1) GRAMMER.

OBJECTIVE: Implement the given grammer in JFLAP and perform stack implementation.

i. Grammer:

E->E+T|t

T->T*F|F

F->(E)|a

Augumented grammer

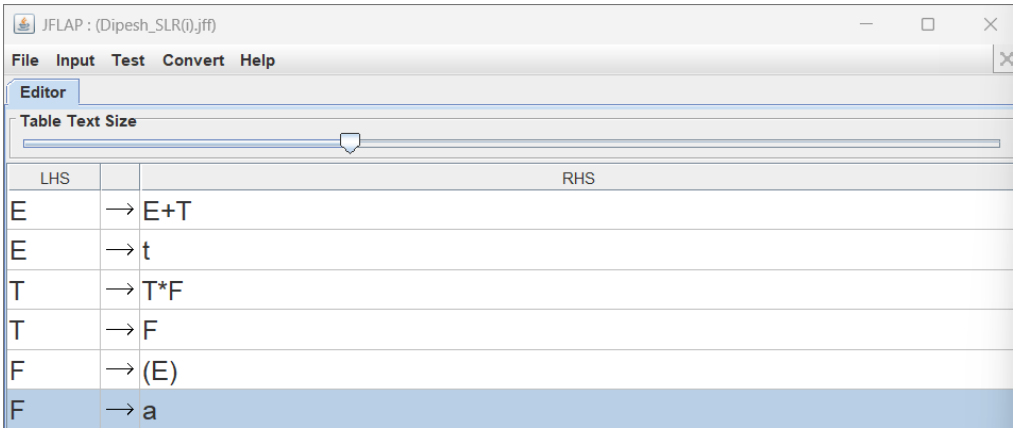
E'->E

E->E+T|t

T->T*F|F

F->(E)|a

Input:



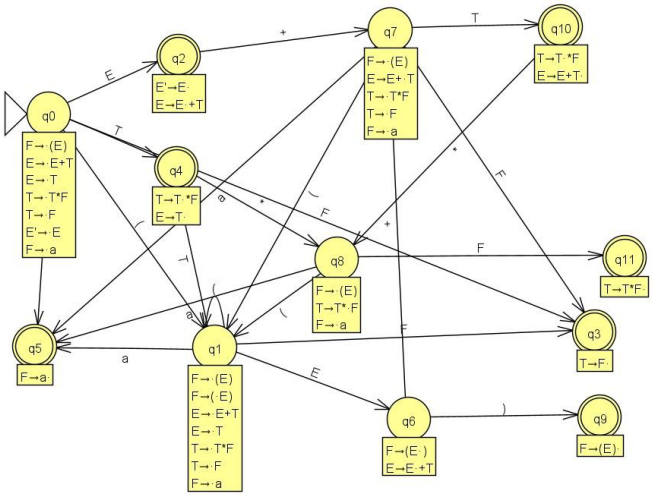
Goto Input -> Build SLR(1) Parse Table



Computing FIRST and FOLLOW

	FIRST	FOLLOW
E	{ a, (}	{ $,), + }
F	{ a, (}	{ $,), *, + }
T	{ a, (}	{ $,), *, + }

Cannonical Collection



SLR Parsing Table

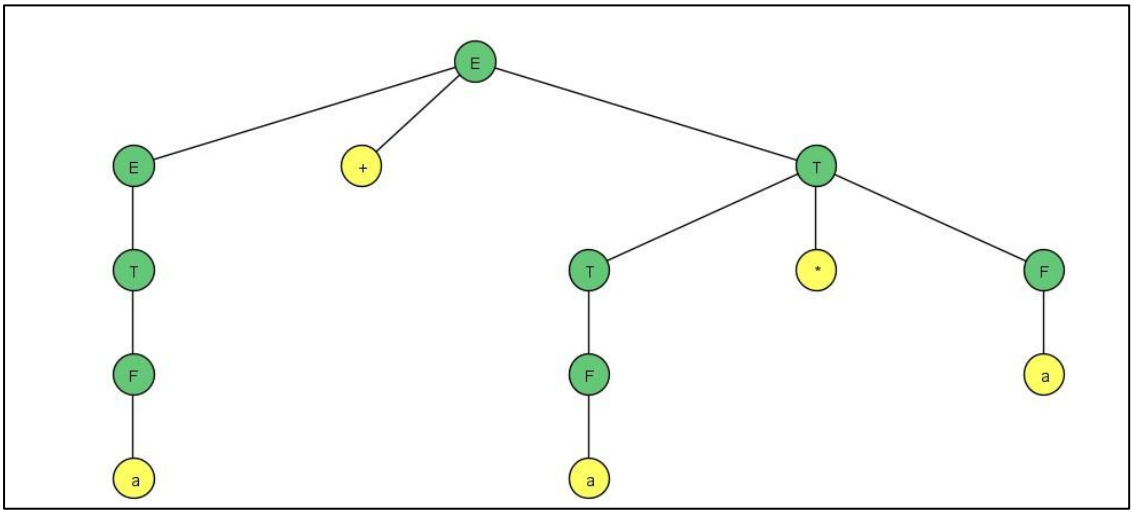
	(	)	*	+	a	$	E	F	T
0	s1				s5		2	3	4
1	s1				s5		6	3	4
2				s7		acc			
3		r4	r4	r4		r4			
4		r2	s8	r2		r2			
5		r6	r6	r6		r6			
6		s9		s7					
7	s1				s5			3	10
8	s1				s5			11	
9		r5	r5	r5		r5			
10		r1	s8	r1		r1			
11		r3	r3	r3		r3			

Stack implementation: a+a*a

STACK	INPUT	ACTION
$0	a+a*a$	Shift → S5
$0a5	+a*a$	Reduce by F→a
$0F3	+a*a$	Reduce by T→F
$0T4	+a*a$	Reduce by E→T
$0E2	+a*a$	Shift → S6
$0E2+7	a*a$	Shift → S5
$0E2+7a5	a*a$	Reduce by F→a
$0E2+73	*a$	Reduce by T→F
$0E2+7T10	*a$	Shift → S7
$0E2+7T10*8	a$	Shift → S5
$0E2+7T10*8a5	$	Reduce by F→a
$0E2+7T10*8F11	$	Reduce by T→T*F
$0E2+7T10	$	Reduce by E→E+T
$0E2	$	Accepted.

Parse tree input:

a+a*a



Derivation Table:

Start | Step | Derivation Table

Input: a+a*a

Input Remaining: $

Stack: E0

Input Field Text Size (For optimization, move one of the window size adjustors around this window after resizing the text fields)

Table Text Size

	a+a*a
F→a	F+a*a
T→F	T+a*a
E→T	E+a*a
F→a	E+F*a
T→F	E+T*a
F→a	E+T*F
T→T*F	E+T
E→E+T	E

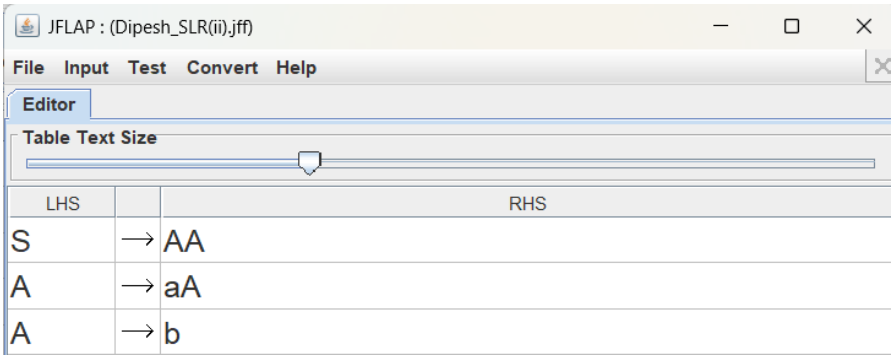
ii. Grammar:

S->AA A-
>Aa a->b

Augmented grammar

S'->S
S->AA
S->Aa
S->b

Input in jflap:

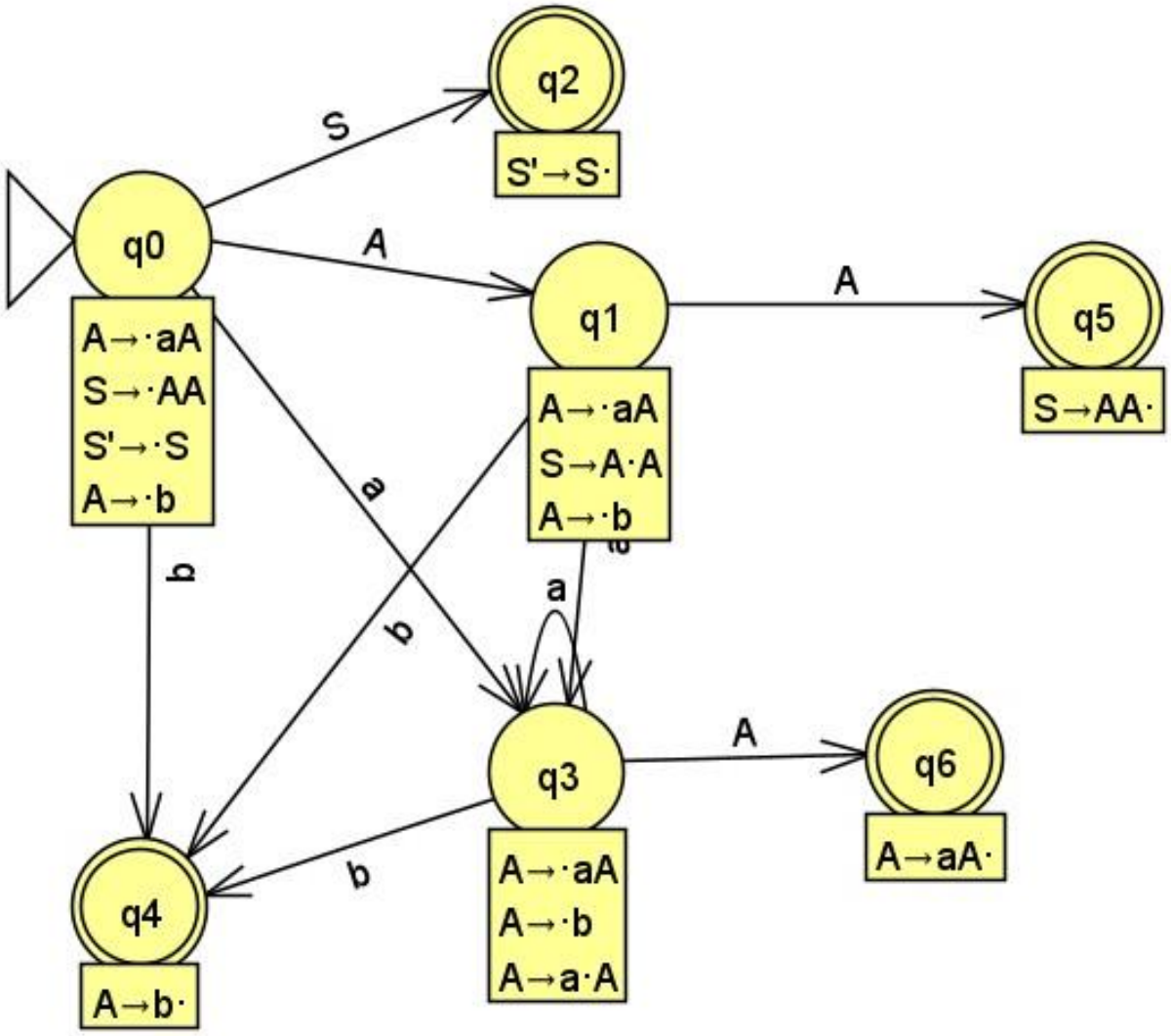


Computing FIRST and FOLLOW

Parse table complete. Press "parse" to use it.

	FIRST	FOLLOW
A	{ a, b }	{ a, b, $ }
S	{ a, b }	{ $ }

Cannonical Collection:



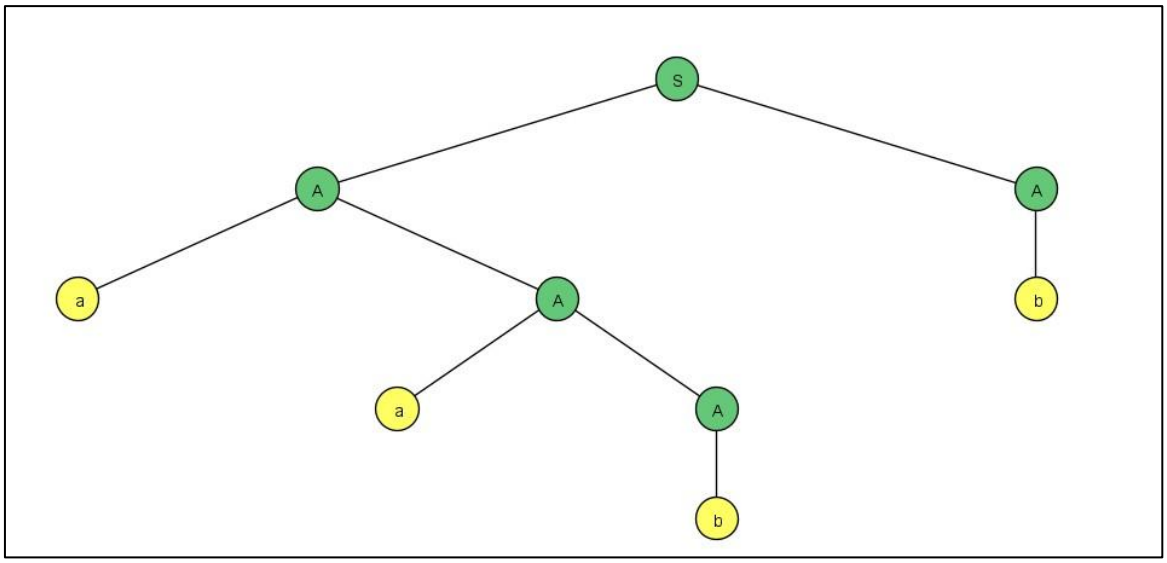
SLR(1) parsing table:

	a	b	$	A	S
0	s3	s4		1	2
1	s3	s4		5	
2			acc		
3	s3	s4		6	
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

Stack Implementation for aabb

STACK	INPUT	ACTION
$0	aabb$	Shift ⇒ S3
$0a3	abb$	Shift ⇒ S3
$0a3a3	bb$	Shift ⇒ S4
$0a3a3b4	b$	Reduce by A→b
$0a3a3A6	b$	Reduce by A→aA
$0a3A6	b$	Reduce by A→aA
$0A1	b$	Shift ⇒ S4
$0A1b4	$	Reduce by A→b
$0A1A5	$	Reduce by S→AA
$0S2	$	Accept

Parse Tree Input: aabb



Derivation table

Table Text Size

	aabb
A→b	aaAb
A→aA	aAb
A→aA	Ab
A→b	AA
S→AA	S

LAB: 14

TITLE: IMPLEMENTATION OF LL(1) GRAMMER.

OBJECTIVE: Implement the given grammar in JFLAP and perform stack implementation.

i. Grammar:

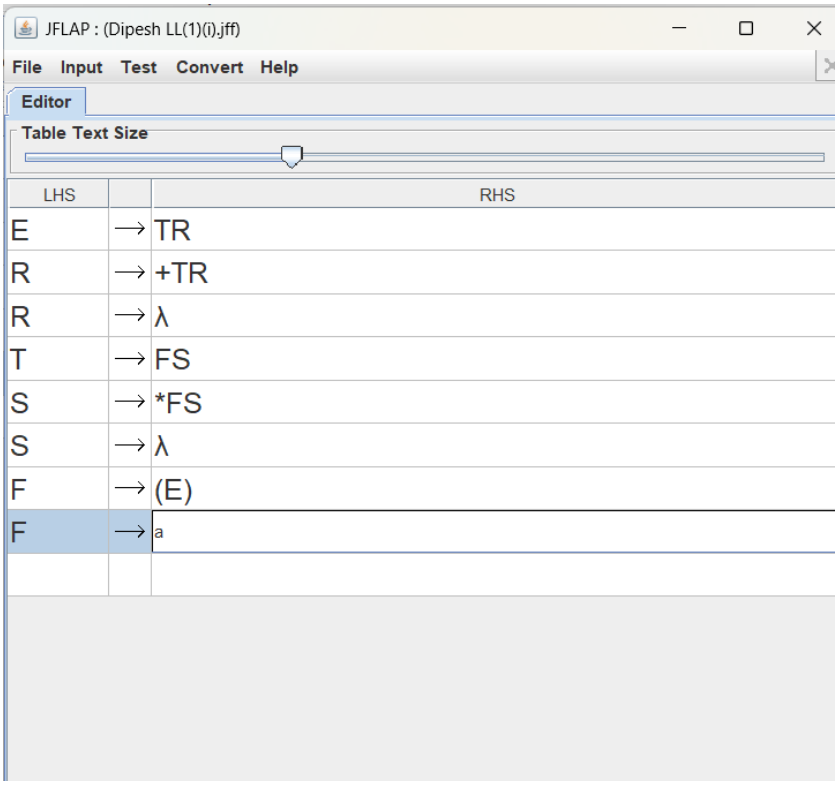
E->E+T |T
T->T*F |F
F->(E) |a

Removing left recursion

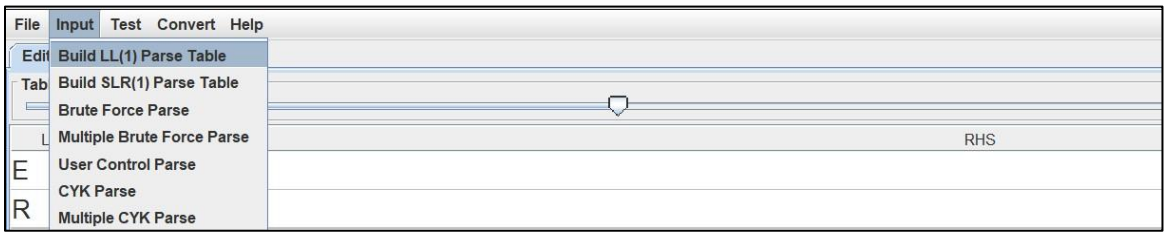
E->TR
R->+TR | λ
T->FS

S->*FS| λ
F->(E) | a

Input



GOTO: Input -> Build LL(1) Parsing



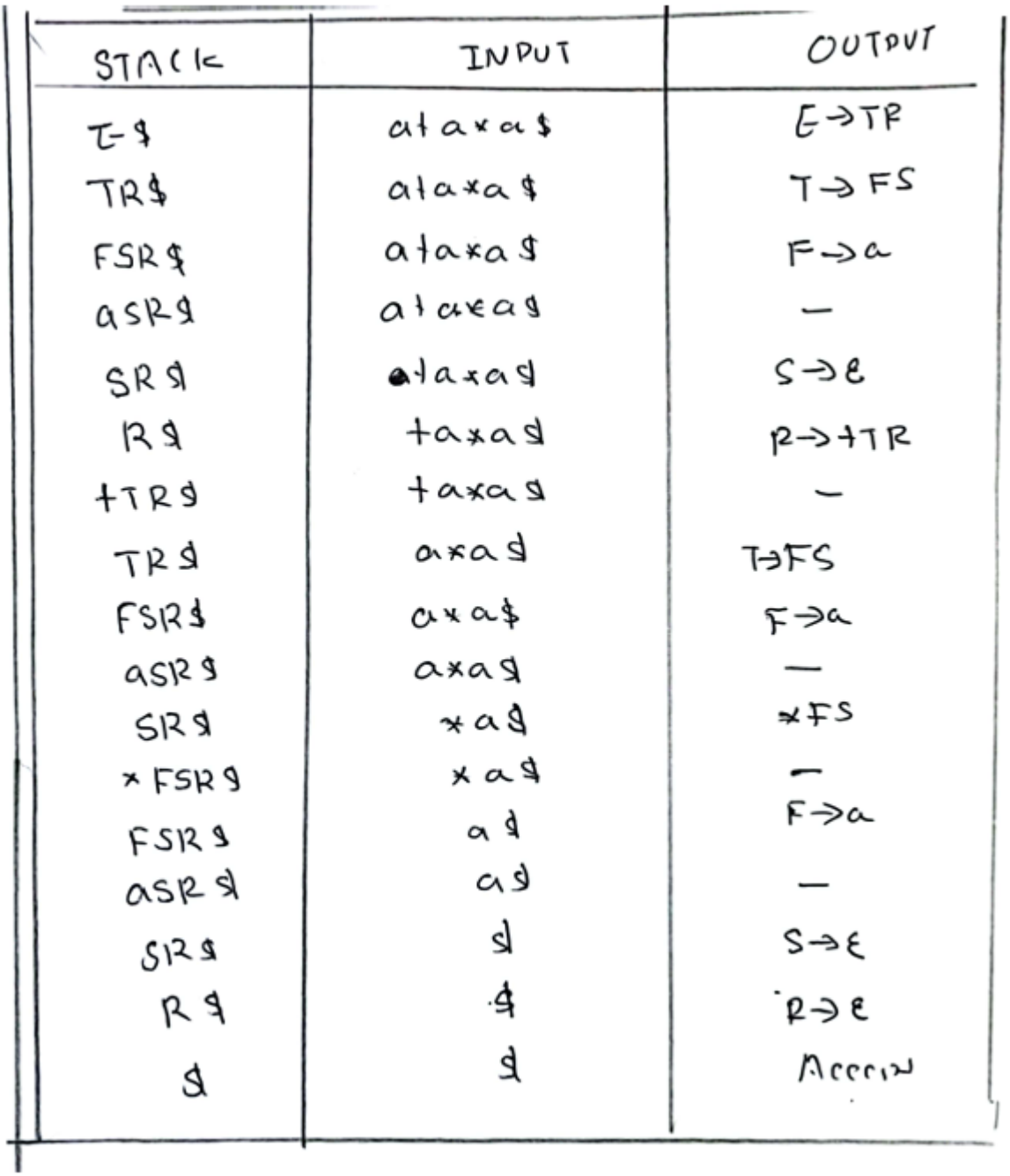
Computing first and follow:

	FIRST	FOLLOW
E	{ a, (}	{ $,) }
F	{ a, (}	{ $,), *, + }
R	{ λ, + }	{ $,) }
S	{ λ, * }	{ $,), + }
T	{ a, (}	{ $,), + }

LL(1) Parsing Table:

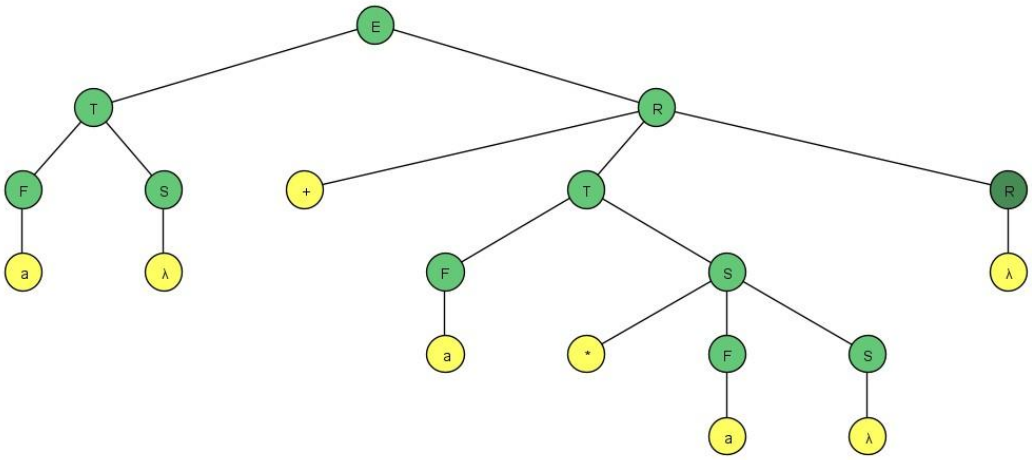
	(	)	*	+	a	$
E	TR				TR	
F	(E)				a	
R		λ		+TR		λ
S		λ	*FS	λ		λ
T	FS				FS	

Stack Implementation: a+a*a



STACK	INPUT	OUTPUT
E$	a+a*a$	E→TR
TR$	a+a*a$	T→FS
FSR$	a+a*a$	F→a
aSR$	a+a*a$	—
SR$	+a*a$	S→ε
R$	+a*a$	R→+TR
+TR$	+a*a$	—
TR$	a*a$	T→FS
FSR$	a*a$	F→a
aSR$	a*a$	—
SR$	*a$	*FS
*FSR$	*a$	—
FSR$	a$	F→a
aSR$	a$	—
SR$	$	S→ε
R$	$	R→ε
$	$	Accept

Parse Tree



Derivation table

Start | Step | Derivation Table

Input: a+a*a

Input Remaining: $

Stack:

Input Field Text Size (For optimization, move one of the window size adjustors around this window after resizing the text fields)

Table Text Size

	E
E→TR	TR
T→FS	FSR
F→a	aSR
S→λ	aR
R→+TR	a+TR
T→FS	a+FSR
F→a	a+aSR
S→*FS	a+a*FSR
F→a	a+a*aSR
S→λ	a+a*aR
R→λ	a+a*a

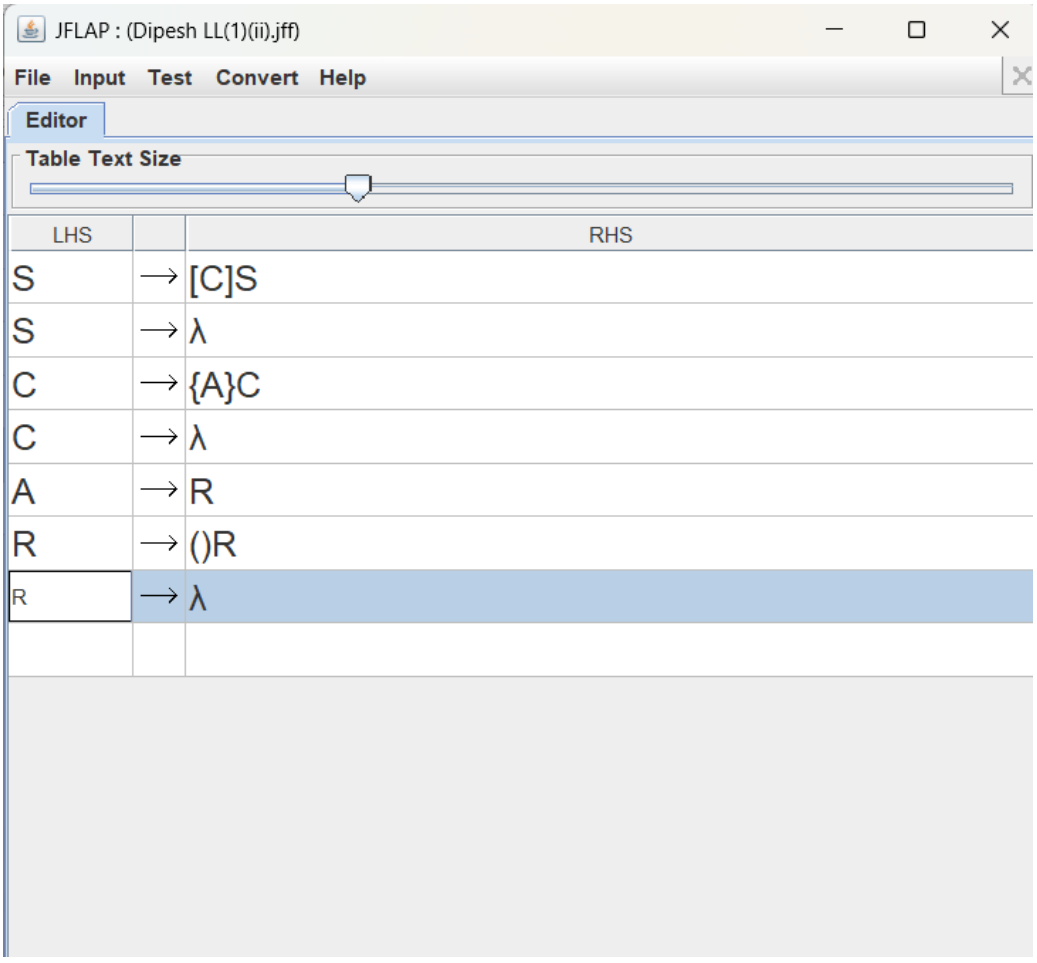
ii. Grammar:

S->[C]S|e
C->{A}C|e
A->A()|E

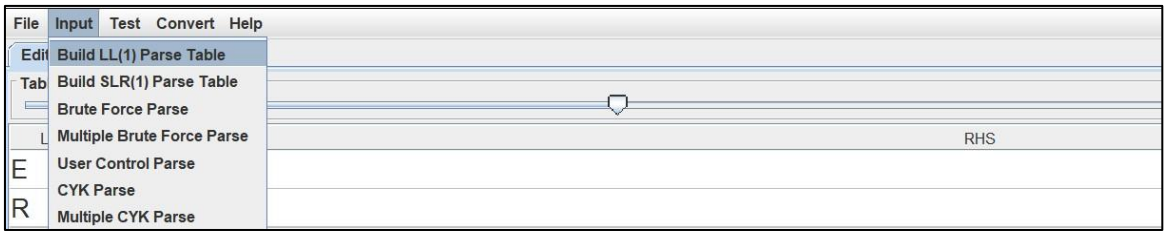
Removing left recursion

S->[C]S
S->LAMDA
C->{A}C
C->LAMDA
A->R
R->()R
R->LAMDA

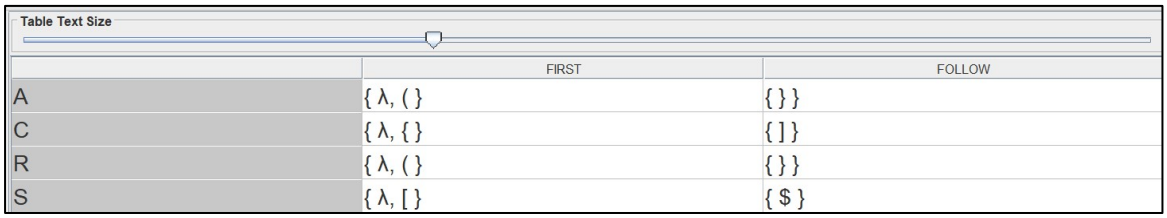
Input in Jflap



GOTO: Input -> Build LL(1) Parse Table



Computing first and follow:



	FIRST	FOLLOW
A	{ λ, (}	{) }
C	{ λ, { }	{] }
R	{ λ, (}	{) }
S	{ λ, [}	{ $ }

LL(1) Parsing Table:

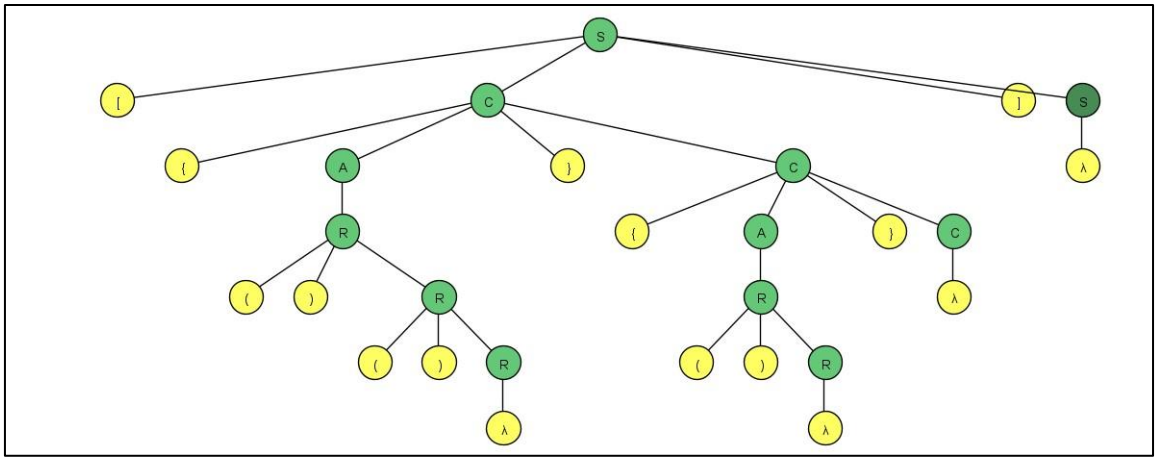
	(	)	[	]	{	}	$
A	R					R	
C				λ	{A}C		
R	()R					λ	
S			[C]S				λ

Stack Implementation of : [{()}{()}]

STACK	INPUT	OUTPUT
S$	[{()}{()}]$	S → [C]S
[C]S$	[{()}{()}]$	—
C]S$	{()}{()}]$	C → {A}C
{A}C]S$	{()}{()}]$	—
A}C]S$	()}{()}]$	A → R
R}C]S$	()}{()}]$	R → ()R
()R}C]S$	()}{()}]$	—
R}C]S$	}{()}]$	R → ε
}C]S$	}{()}]$	—
C]S$	{()}]$	{A}C
{A}C]S$	{()}]$	—
A}C]S$	()}]$	A → R
R}C]S$	()}]$	R → ()R
()R}C]S$	()}]$	—
R}C]S$	}]$	R → ε
}C]S$	}]$	—
C]S$	]$	C → ε
]S$	]$	—
S$	$	S → ε
$	$	Accept.

Parse Tree:

Input: [{()()}{()}]



Derivation Table:

Start Step Derivation Table

Input [{()()}{()}]

Input Remaining $

Stack

Input Field Text Size (For optimization, move one of the window size adjustors around this window after resizing the text fields)

Table Text Size

	S
S→[C]S	[C]S
C→{A}C	[{A}C]S
A→R	[{R}C]S
R→()R	[{()R}C]S
R→()R	[{()()R}C]S
R→λ	[{()()}C]S
C→{A}C	[{()()}{A}C]S
A→R	[{()()}{R}C]S
R→()R	[{()()}{()R}C]S
R→λ	[{()()}{()}C]S
C→λ	[{()()}{()}]S
S→λ	[{()()}{()}]

LAB: 15

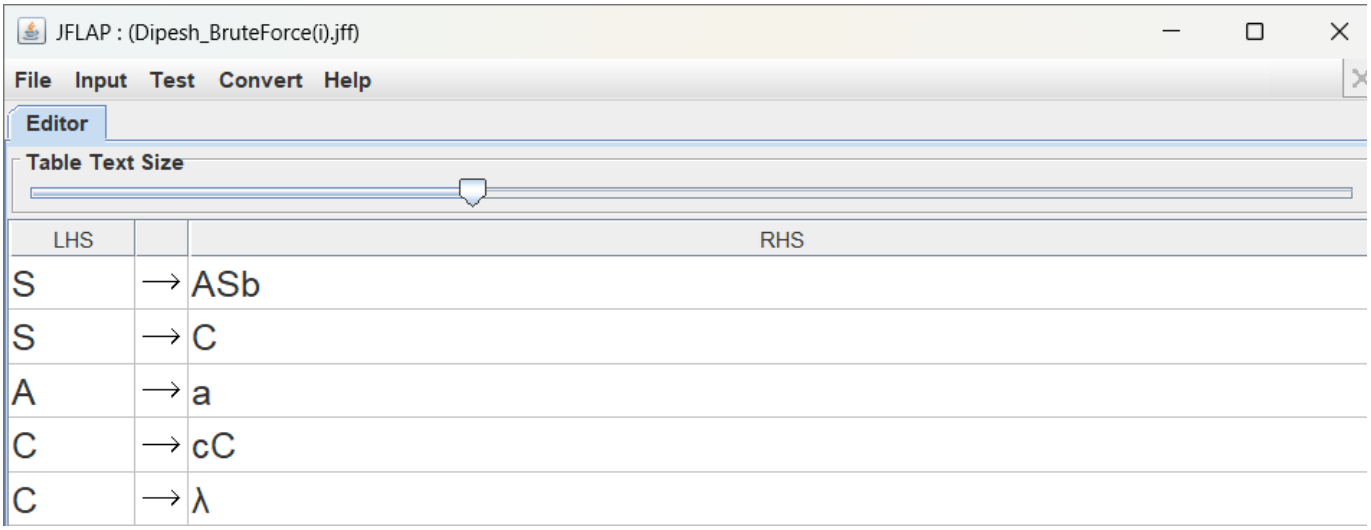
TITLE: IMPLEMENTATION OF BRUTE FORCE PARSER

OBJECTIVE: Illustrate Brute Force Parser of following grammar in JFLAP.

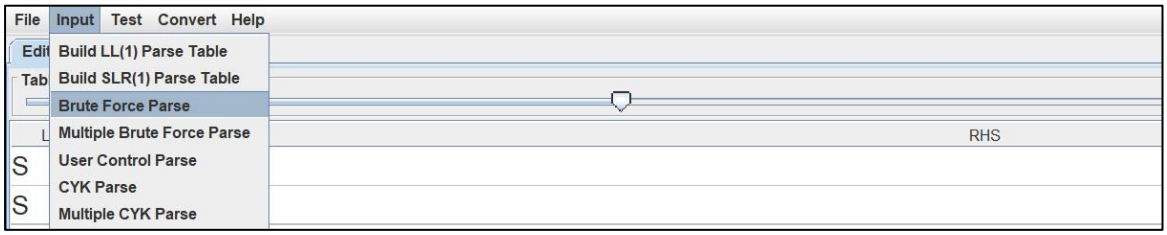
i. Grammar

S->ASb
S->C
A->a
C->cC
C->Epsilon

Input in Jflap:



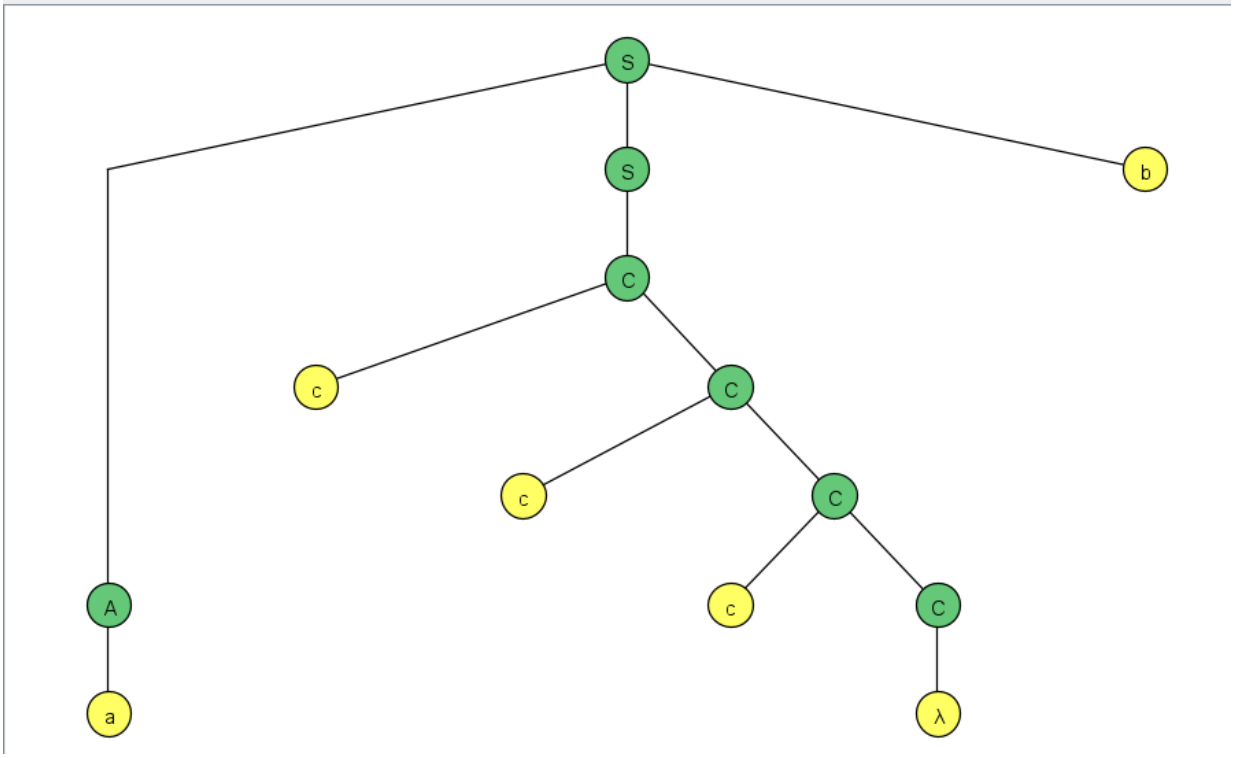
Goto: Input -> Brute Force Parse



Stack Implementation acccb

Stack	INPUT	OUTPUT
S$	acccb$	S→ASb
ASb$	acccb$	A→a
aSb$	acccb$	—
Sb$	cccb$	S→C
Cb$	cccb$	C→cC
cCb$	cccb$	—
Cb$	ccb$	C→cC
cCb$	ccb$	—
Cb$	cb$	C→cC
cCb$	cb$	—
Cb$	cb$	C→ε
b$	b$	—
$	$	Accepted

Parse tree : acccb



Derivation Table

Table Text Size

	S
S→ASb	ASb
S→C	ACb
C→cC	AcCb
C→cC	AccCb
C→cC	AcccCb
A→a	acccCb
C→λ	acccb

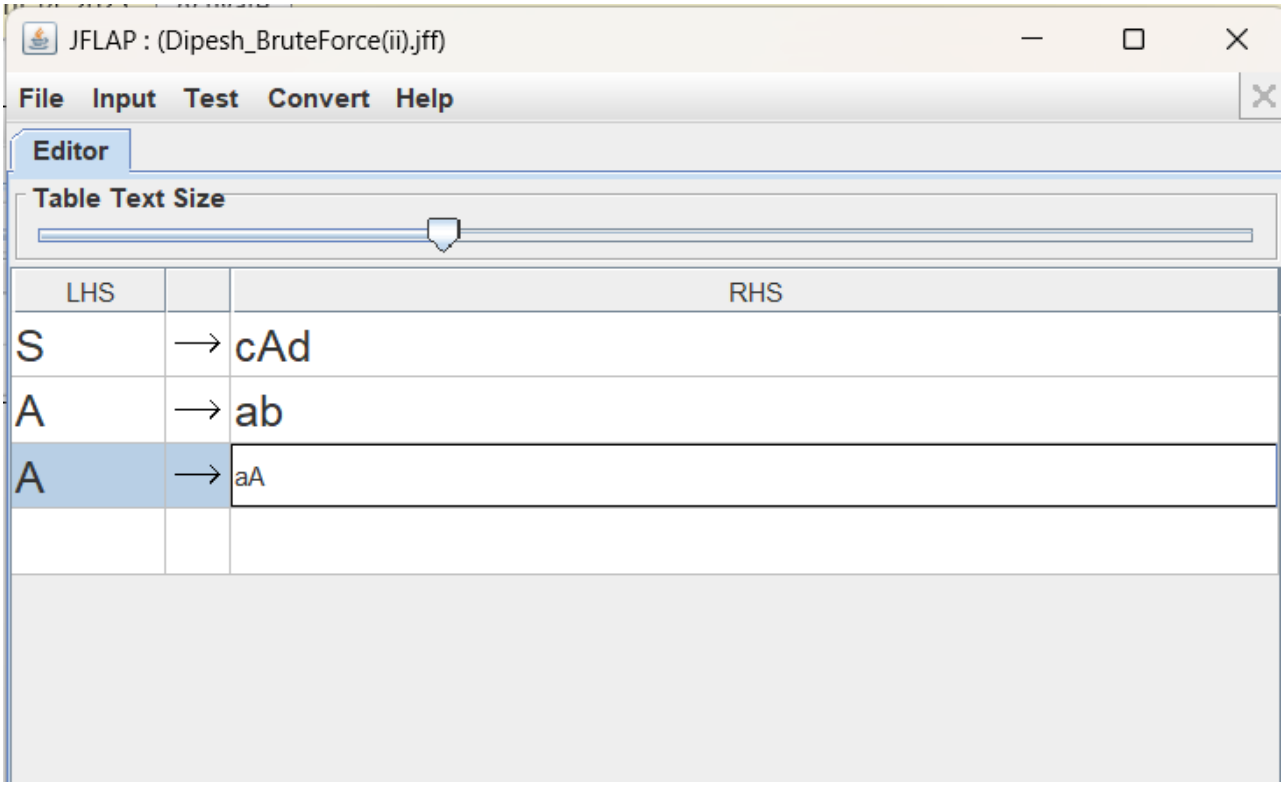
ii. Grammar:

S->cAd

A->ab

A->d

Input in Jflap



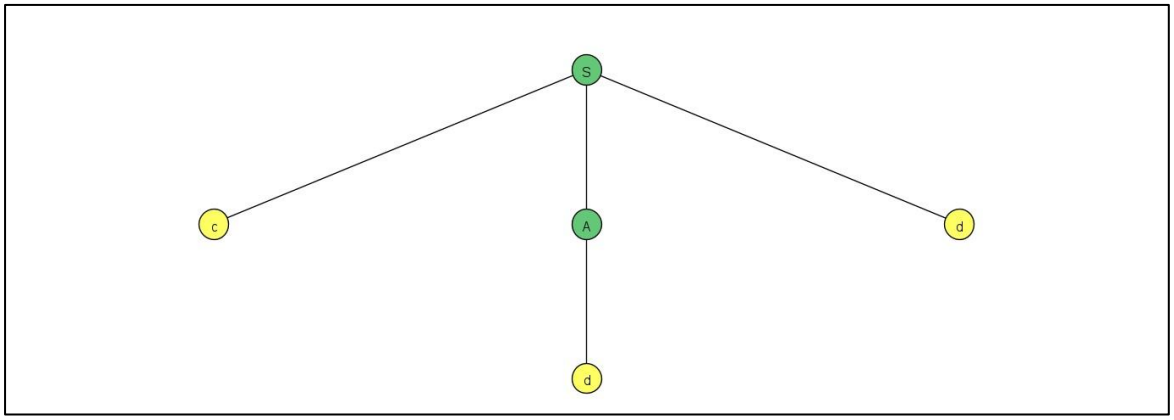
Goto: Input -> Brute Force Parse



Stack Implementation for caaaabd

Stack	Input	Output
S$	caaaabd$	S→cAd
cAd$	caaaabd$	—
Ad$	aaaabd$	A→aA
aAd$	aaaabd$	—
Ad$	aaabd$	A→aA
aAd$	aaabd$	—
Ad$	aabd$	A→aA
aAd$	aabd$	—
Ad$	abd$	A→ab
abd$	abd$	—
bd$	bd$	—
d$	d$	—
$	$	Accepted

Parse Tree



Derivation Tree

Table Text Size

	S
S→cAd	cAd
A→d	cdd